Sync Guide

Light-client synchronisation: compact blocks, the service, and the scanning pipeline

m@rek.onl

Abstract

Assuming the Orchard protocol of the *Ironwood Guide* and the wallet layer of the *Wallet Guide*, this volume of *The Zcash Arboretum* documents how a light client synchronises: the compact-block format (ZIP-307) and its deployed divergences, the light-client service surface as lightwalletd and Zaino implement it, the scanning pipeline of librustzcash with its verify-first reorg discipline, commitment-tree synchronisation, and — stated without euphemism — what the server learns about its clients. The wire format is deployed but underspecified; where the draft ZIP and the deployed implementations disagree, this volume documents the deployment and flags the divergence.

Contents

1 Compact blocks (ZIP-307)

A wallet that trusts no server detects its notes by trial-decrypting every shielded output on the chain (*Ironwood Guide*, §"Transmission and detection of a note: encryption, trial decryption, and synchronisation"). That obligation costs twice: one key agreement of computation per output, and the bandwidth of downloading every output. ZIP 307 (*Light Client Protocol for Payment Detection*) attacks the bandwidth half. A *compact block* is "a packaging of ONLY the data from a block needed to" (1) detect a payment to one's shielded address, (2) detect a spend of one's notes, and (3) update witnesses for new spend proofs (ZIP 307, "Compact Stream Format"). The deployed protocol widens this beyond ZIP 307's Sapling-only scope to every shielded pool, and adds a fourth purpose: detecting spends of the coins of the wallet's transparent addresses (`compact_formats.proto`, header comment, lines 21–26). The computational half survives compaction — every compact output must still be trial-decrypted — and that residual linear scan is among the costs whose removal motivates the successor design (*Tachyon Guide*, §"What the fold removes").

1.1 Normative status and wire-format history

ZIP 307 carries Status: Draft, covers Sapling only, and nowhere mentions Orchard, `ChainMetadata`, or transparent compact data (`zip-0307.rst`, header and whole file). Its message definitions are stale relative to deployment: the ZIP text defines `CompactBlock{BlockID id = 1; vtx = 3}` and `CompactTx{txIndex, txHash, spends, outputs}`, with element messages named `CompactSpend` and `CompactOutput` (ZIP 307, "Compact Stream Format"), whereas the deployed messages of §1.2 use different names, field numbers, and fields. The renames to `CompactSaplingSpend`/`CompactSaplingOutput` date to lightwallet-protocol v0.3.1 (2021-12-09), which also added `CompactOrchardAction` (lightwallet-protocol, `CHANGELOG.md`).

The canonical proto files live in the `zcash/lightwallet-protocol` repository and are vendored — git subtree or symlink — into lightwalletd, librustzcash (the files under `zcash_client_backend/proto/` are symlinks into `lightwallet-protocol/walletrpc/`), and Zaino. The repository's README states that it "IS not a specification" and defers to ZIP 307 (lightwalletd, `lightwallet-protocol/README.md`) — which is itself stale. No current document therefore normatively specifies the deployed wire format. Per our conventions we classify the compact-block format *deployed but underspecified*: we cite lightwallet-protocol tagged releases as the de facto definition, and flag each point where ZIP text and deployment disagree (§1.8).

The three deployed codebases vendor three different revisions: lightwalletd v0.4.1 (`protoVersion` present, no Ironwood); librustzcash main v0.5.0 (`protoVersion` removed and reserved; Ironwood fields present); Zaino an intermediate snapshot carrying *both* the Ironwood fields and `protoVersion` (diff of the three `compact_formats.proto` copies). We flag the skew, and a changelog-date contradiction, in §1.8. Deployed-behaviour claims in this section are verified against librustzcash `e30517e4`, lightwalletd `61fee32e`, Zaino `0e057e22`, and zips `6961098` (checkouts of 2026-07-14).

1.2 The deployed messages

The messages live in the protobuf package `cash.z.wallet.sdk.rpc` and declare `proto3` syntax (`compact_formats.proto`, lines 5–11). Proto3 semantics make every field optional: an absent field decodes to zero or empty, indistinguishable from an explicit zero. The scanner must defend against this — the default-zero tree sizes of §1.5 are the load-bearing case.

Tag	Date	Change
v0.3.1	2021-12-09	`CompactOrchardAction` added; `CompactSpend/CompactOutput` renamed `CompactSaplingSpend/CompactSaplingOutput`
v0.3.2	2021-12-09	`CompactOrchardAction.encCiphertext` renamed `ciphertext`
v0.3.5	2023-07-03	`ChainMetadata` and `CompactBlock.chainMetadata` added; `GetSubtreeRoots/SubtreeRoot` added; nullifier RPCs added; `CompactSaplingOutput.epk` renamed `ephemeralKey`
v0.4.0	2025-12-03	`CompactTxIn/TxOut` and `vin/vout` added; `BlockRange.poolTypes` added; `LightdInfo.lightwalletProtocolVersion` added; `CompactTx.hash` renamed `txid` (serialization-compatible)
v0.4.1	2026-02-20	nullifier RPCs designated deprecated in the proto
v0.5.0	2026-06-30	Ironwood fields added; `protoVersion` removed, field number 1 and name reserved

Table 1: Wire-format history of lightwallet-protocol (`CHANGELOG.md`, librustzcash-vendored copy).

```
message CompactBlock {
  uint32 protoVersion = 1;                          removed in v0.5.0; number and
                                                    name reserved
  uint64 height = 2;                                block height
  bytes hash = 3;                                   block hash, 32 bytes
  bytes prevHash = 4;                               parent block hash, 32 bytes
  uint32 time = 5;                                  Unix epoch time
  bytes header = 6;                                 "should always be unset
                                                    (empty)"
  repeated CompactTx vtx = 7;                       compact transactions
  ChainMetadata chainMetadata = 8;                  end-of-block tree sizes
}
message ChainMetadata {
  uint32 saplingCommitmentTreeSize = 1;             Sapling tree size at end of block
  uint32 orchardCommitmentTreeSize = 2;             Orchard tree size at end of block
  uint32 ironwoodCommitmentTreeSize = 3;            v0.5.0 only
}
message CompactTx {
  uint64 index = 1;                                 position within the block
  bytes txid = 2;                                   32 bytes, protocol byte order
  uint32 fee = 3;                                   unpopulated in practice
  repeated CompactSaplingSpend spends = 4;
  repeated CompactSaplingOutput outputs = 5;
  repeated CompactOrchardAction actions = 6;
  repeated CompactTxIn vin = 7;                     coinbase input omitted
  repeated TxOut vout = 8;
  repeated CompactOrchardAction ironwoodActions = 9;    v0.5.0 only
}
```

The container messages, from the deployed protos (lightwalletd vendored copy, `walletrpc/compact_formats.proto`, lines 14–80; librustzcash copy, lines 14–86):

- Field `header` "should always be unset (empty)" per the proto comment (lines 27–30). ZIP 307's block header validation — Equihash, `bits`, difficulty — is therefore unimplementable against deployed servers; the ZIP itself flags the section as "only partially implemented: currently only `prevHash` is checked" (ZIP 307, "Block header validation"). Client-side, only `prevHash` and height continuity are enforced (§1.5). What a header-verifying

client would gain from the ZIP-221 chain-history commitment is the *FlyClient Guide*'s subject.

- Field `protoVersion` never varies on the wire. lightwalletd never sets it, the assignment commented out as a `TODO` (`parser/block.go`, line 112); deployed Zaino serves `0` on every path (`zaino-state`, `compact_block.rs` line 285, `legacy.rs` line 1113), which for a proto3 scalar is byte-identical to lightwalletd's absent field—its lone non-zero literal (`zaino-fetch`, `src/chain/block.rs`, line 218) sits in a parse-error `Debug` string and never reaches serialization; v0.5.0 removed the field, reserving number 1 and the name (librustzcash copy, lines 33–36). The librustzcash scanner never reads it. Flagged in §1.8.

- Message `ChainMetadata` states each pool's note commitment tree size *as of the end of this block*, as `uint32` (`compact_formats.proto`, lines 14–17): the tree holds at most 2^{32} leaves, and the scanner maps arithmetic overflow to `ScanError::TreeSizeOverflow` (zcash_client_backend, `src/scanning.rs`, lines 649–654). Section 1.5 shows how these sizes turn a compact block into note positions.

- Field `txid` carries the 32-byte transaction identifier in protocol byte order; the proto comment mandates that it "MUST NOT be reversed or hex-encoded" — the reversed hex form is display-only — citing protocol spec §7.1.1, "Transaction Identifiers" (`compact_formats.proto`, lines 52–57).

- The `fee` field is documented "present if server can provide", but neither deployed server populates it: lightwalletd leaves it commented out with `TODO: calculate fees` (`parser/transaction.go`, line 407) and Zaino hardcodes `fee: 0` (`zaino-fetch`, `src/chain/transaction.rs`, line 369). Clients cannot rely on it. For a transaction with no transparent inputs the proto comment gives the client-side reconstruction (`compact_formats.proto`, lines 59–64):

  $$\text{fee} = \text{valueBalanceSapling} + \text{valueBalanceOrchard} + \sum \text{vPubNew} - \sum \text{vPubOld} - \sum \text{tOut},$$

  with valueBalanceIronwood joining the sum in v0.5.0.[1]

- Field `vin` omits the coinbase transaction's single null-outpoint input; a light client recognises a coinbase `CompactTx` by `index` = 0, coinbase being always first in a block. Server lightwalletd implements exactly this, skipping `vin` when `index` = 0 (`compact_formats.proto`, lines 70–76; `parser/transaction.go`, lines 398–403). The transparent element messages are `CompactTxIn{prevoutTxid, prevoutIndex}` and `TxOut{value, scriptPubKey}` (`compact_formats.proto`, lines 82–104).

The shielded elements. Each shielded element message keeps only the detection-relevant bytes of its full on-chain description.

A `CompactSaplingSpend` retains only the 32-byte nullifier of the 384-byte Spend description (cv 32, anchor 32, nullifier 32, rk 32, zkproof 192, spendAuthSig 64) — the one field needed to detect a spend of one's own notes; ZIP 307 quotes this as a 90% bandwidth improvement (ZIP 307, "Spend Compression"). A `CompactSaplingOutput` retains `cmu`, the ephemeral key, and the first 52 bytes of encCiphertext, "Total size is 116 bytes" per the proto comment, against 580 bytes of ciphertext plus the 32-byte ephemeral key streamed in full form — ZIP 307's 80% reduction (ZIP 307, "Output Compression"; lightwalletd `compact_formats.proto`, lines 114–122; full encodings per protocol spec §§7.3–7.4). A `CompactOrchardAction` serves both roles of an Action in one message: its `nullifier` detects spends of the input note, while `cmx`, `ephemeralKey`, and `ciphertext` detect and place the output note (`compact_formats.proto`, lines 124–130; full Action encoding per protocol spec §7.5).

[1] Type `uint32` additionally caps the representable fee at $2^{32} - 1$ zatoshi (≈ 42.9 ZEC); the proto comment does not remark on this.

Message	Fields (bytes)	Payload	Full form	Saving
`CompactSaplingSpend`	`nf` (32)	32	384	∼90%
`CompactSaplingOutput`	`cmu` (32), `ephemeralKey` (32), `ciphertext` (52)	116	580 + 32	∼80%
`CompactOrchardAction`	`nullifier` (32), `cmx` (32), `ephemeralKey` (32), `ciphertext` (52)	148	—	—

Table 2: Compact shielded elements (`compact_formats.proto`, lines 106–130). The Sapling payload and saving figures are ZIP 307's ("Spend Compression", "Output Compression"); the 148-byte Orchard total is arithmetic, not stated in the sources. Payload figures count raw bytes; on the wire, a `CompactSaplingSpend` encodes to 34 bytes (36 as a `CompactTx.spends` element), a `CompactSaplingOutput` to 122 (124), and a `CompactOrchardAction` to 156 (159 — its 156-byte payload needs a two-byte length varint). Field headers add 2 bytes per 32-byte field and 2 per 52-byte ciphertext. Computed by script, proto-encoding every compact element of mainnet block 3,412,934 (3 spends, 8 outputs, 10 actions; `GetBlock`, lightwalletd v0.4.19, 2026-07-15); the sizes are block-independent because all field lengths are fixed.

1.3 Omissions and fetch-on-detect

Compact blocks omit the 512-byte memo, the 16-byte AEAD tag, the 80-byte outCiphertext, cv, rk, anchors, zkproofs, and signatures (ZIP 307, "Transaction privacy"; sizes per zcash_note_encryption 0.4.1, `src/lib.rs`, lines 44–57). Recovering a memo, or spend information via the outgoing ciphertext, thus requires downloading the full transaction — deployed as the `GetTransaction` RPC. ZIP 307 attaches three requirements to that fetch: the client SHOULD obscure its interest by also downloading numerous uninteresting transactions, SHOULD download all transactions of any block it touches, and MUST convey the privacy reduction to the user (ZIP 307, "Transaction privacy").

The deployed pipeline expresses the fetch as data: scanning emits `TransactionDataRequest::Enhancement(txid)` — "download of complete raw transaction data" — which the caller satisfies via `GetTransaction` and feeds to `wallet::decrypt_and_store_transaction` (zcash_client_backend, `src/data_api.rs`, lines 1185–1207, 2210). The crate's own sync loop documents that it does not yet perform enhancement (zcash_client_backend, `src/sync.rs`, lines 1–10).

1.4 Compact trial decryption

The 52-byte `ciphertext` field is the prefix of encCiphertext that encrypts exactly the *compact note plaintext*: lead byte (1) ‖ diversifier d (11) ‖ value v (8) ‖ rseed (32), so COMPACT_NOTE_SIZE = 52; the full plaintext appends the 512-byte memo, and the full ciphertext a 16-byte tag, giving 580 bytes (zcash_note_encryption 0.4.1, `src/lib.rs`, lines 44–57). ZIP 307's rationale: these bytes "contain the contents and opening of the note commitment, which is all of the data needed to spend the note and to verify that the note is spendable" (ZIP 307, "Output Compression"). The *Ironwood Guide* constructs compact decryption for Orchard in §"Transmission and detection of a note: encryption, trial decryption, and synchronisation"; we do not repeat the construction, and record here only the deployed mechanics and the integrity argument.

Function `try_compact_note_decryption` (doc-comment: "Implements the procedure specified in ZIP 307") proceeds as follows (zcash_note_encryption 0.4.1, `src/lib.rs`, lines 567–604):

$$K := \mathsf{KDF}\big(\mathsf{KA.Agree}(\mathsf{ivk}, \mathsf{epk}),\, \mathsf{ephemeralKey}\big),$$
$$\mathsf{np} := \mathsf{ciphertext}[0..52] \oplus \mathsf{ChaCha20}_K\big(\text{nonce} = 0^{96}\big)[64..116],$$

a bare ChaCha20 keystream with zero nonce, sought to byte offset 64 — skipping keystream block 0, which the full AEAD reserves for the Poly1305 key (`src/lib.rs`, lines 590–593). The candidate np is parsed as a compact note plaintext (no memo), then `check_note_validity` accepts iff

(i) the plaintext parses, and

(ii) the recomputed commitment matches the on-chain one, $\mathsf{Extract}\big(\mathsf{NoteCommit}_{\mathsf{rcm}}(\ldots)\big) = \mathsf{cmu}$ (respectively cmx), and

(iii) per ZIP-212, the ephemeral secret esk re-derived from rseed reproduces the published key: the byte encoding of $\mathsf{KA.DerivePublic}(\mathsf{esk}, \mathsf{g_d})$ equals `ephemeralKey`, compared in constant time

(`src/lib.rs`, lines 524–604). Condition (ii) is the integrity substitute: truncation at 52 bytes discards the AEAD tag, so a compact decryption cannot be authenticated via Poly1305; ZIP 307 instead recomputes the note commitment from the decrypted plaintext and compares against the on-chain cmu, itself authenticated by the binding signature over the transaction (ZIP 307, "Output Compression"). Condition (iii) is the deployed strengthening beyond the tag: it defeats a mauled ephemeral key (*Ironwood Guide*, §"Transmission and detection of a note").

Lead byte and the ZIP-212 grace period. The lead byte is `0x01` pre-Canopy and `0x02` under ZIP-212; deployed code switches on `zip212_enforcement`: during the grace period of 32,256 blocks after Canopy activation both lead bytes are accepted, and thereafter only `0x02` (zcash_primitives, `src/transaction/components/sapling.rs`, lines 27–40; zcash_protocol, `src/consensus.rs`, line 681). ZIP 307's pseudocode disagrees with itself here: its two "[Canopy onward]" lines are *both* conditioned on $\mathsf{height} < \mathsf{CanopyActivationHeight} + \mathsf{ZIP212GracePeriod}$, the first rejecting $\mathsf{leadByte} \notin \{\texttt{0x01}, \texttt{0x02}\}$ and the second rejecting $\mathsf{leadByte} \neq \texttt{0x02}$ (ZIP 307, "Scanning for relevant transactions", `zip-0307.rst`, lines 275–276). Read literally, the pair rejects `0x01` *during* the grace period and constrains nothing after it — the inverse of the intent; the second condition should read $\geq$. Deployed code implements the intended semantics. The defect is tracked in `research/upstream-defects.md`, item 1 (ZIP 307: second Canopy-onward lead-byte condition rejects `0x01` during the grace period; verified 2026-07-15 at zips 69610984). We flag the divergence in §1.8.

1.5 Scanning: spends, positions, and tree sizes

Spend detection. The scanner parses every compact nullifier field up front: each `CompactSaplingSpend.nf` and each `CompactOrchardAction.nullifier`. A malformed length yields `ScanError::EncodingInvalid` rather than a panic. The parsed nullifiers are compared against the wallet's known-unspent nullifiers in constant time (`find_spent`); unmatched nullifiers are retained in per-block nullifier maps (zcash_client_backend, `src/scanning/compact.rs`, lines 312–390; `src/scanning.rs`, lines 826–828).

Tree building and positions. Every cmu and cmx in the block — not only the wallet's — is appended to the wallet's note commitment tree with `Retention` marks, and each detected note is marked at its position (zcash_client_backend, `src/scanning.rs`, lines 786–824). The position is computed arithmetically, never by replaying the chain:

$$\text{pos}(\text{output } i \text{ of } tx) \;=\; \text{start\_tree\_size} \;+\; \text{offset}(tx) \;+\; i,$$

where $\text{offset}(tx)$ counts the outputs of earlier transactions in the block (`PositionTracker::{sapling,orchard}_note_position`). The shardtree that consumes these marks is constructed

in the *Orchard Guide*, §"Proof of ownership of *some* valid note: the commitment tree"; the observation that a light wallet reads positions directly off the block appears in §"Transmission and detection of a note".

Positions are load-bearing, not merely convenient: the `TreeSizeUnknown` doc-comment records that without the tree size it is "impossible to construct the nullifier for a detected note" — the Sapling nullifier depends on the note's position, via $\rho = \mathsf{MixingPedersenHash}(\mathsf{cm}, \mathsf{NotePosition})$; Orchard's does not, but both pools need positions for witnesses (zcash_client_backend, `src/scanning.rs`, lines 633–639; protocol spec, "Computing ρ values and Nullifiers" and "Dummy Notes").

Where the tree size comes from. The start size for a block is resolved by a ladder (`tree_sizes_around`, zcash_client_backend, `src/scanning/compact.rs`, lines 589–696):

$$\text{start}(\text{pool}) := \begin{cases} s_{\text{prior}} & \text{prior } \texttt{BlockMetadata} \text{ known;} \\ s_{\text{final}} - n_{\text{block}} & \texttt{chainMetadata} \text{ present, by checked subtraction;} \\ 0 & \text{block below the pool's activation height;} \\ \bot & \text{otherwise (}\texttt{ScanError::TreeSizeUnknown}\text{),} \end{cases}$$

where s_{final} is the `chainMetadata` end-of-block size and n_{block} the block's own output count. The subtraction is `checked_sub`: underflow yields `ScanError::TreeSizeInvalid`, which is precisely the guard against proto3's default-zero metadata (§1.2). After walking all `CompactTx`s the accumulated position must equal the `chainMetadata` size, else scanning fails with `ScanError::TreeSizeMismatch{given, computed}` (`src/scanning/compact.rs`, lines 257–287, 745–770).

The error taxonomy routes recovery: `TreeSizeMismatch`, `PrevHashMismatch` (`prevHash` against the prior block's hash), and `BlockHeightDiscontinuity` (height $\neq$ prev + 1) report `is_continuity_error()` = true and trigger the truncate-and-rescan reorganisation path (*Wallet Guide*, §"Reorganisations: truncate and rescan"); `EncodingInvalid`, `TreeSizeUnknown`, `TreeSizeInvalid`, and `TreeSizeOverflow` do not (zcash_client_backend, `src/scanning.rs`, lines 602–685). The final tree size also drives checkpointing: the helper `compact_tx_contains_last_{sapling,orchard}_outputs_in_block` compares the running position plus the current transaction's output count against the final size, to decide when to write the block-boundary checkpoint into the commitment tree (`src/scanning/compact.rs`, lines 712–731; call sites 404, 444, 482).

The server side of `ChainMetadata`. Server lightwalletd fills the tree sizes from zcashd's `getblock` verbosity-1 response fields `trees.sapling.size` and `trees.orchard.size` (lightwalletd, `common/common.go`, lines 163–175, 399–400). Upstream, zcashd computes them by anchor lookup (`GetSaplingAnchorAt`/`GetOrchardAnchorAt` on `hashFinalSaplingRoot`/`hashFinalOrchardRoot`) and omits the subfield when the anchor is unavailable — in which case Go's JSON unmarshal leaves `Size` = 0, the default-zero case `TreeSizeInvalid` exists to catch (zcashd, `src/rpc/blockchain.cpp`, lines 279–297; the `trees` field dates to zcashd commit `cf2b6c796`, "rpc: Add trees field to getblock RPC output").

1.6 The scan pipeline: chain state, batching, caching

Function `scan_cached_blocks` requires a `ChainState` — the final Sapling and Orchard tree frontiers as of from_height − 1 — and asserts from_height = `chain_state.height` + 1; both trees have depth 32 (zcash_client_backend, `src/data_api/chain.rs`, lines 504–698). The `ChainState` comes from the server's `GetTreeState` RPC via `TreeState::to_chain_state()`, which hex-decodes and byte-reverses the block hash and parses empty tree fields as empty

trees (src/proto.rs, lines 367–463). Trial decryption of compact outputs is batched through BatchRunners with batch size 100 (src/data_api/chain.rs, line 642).

The proto accessors split their failure modes: block-level convenience accessors panic on malformed server data (CompactBlock::hash and prev_hash if not 32 bytes, height if outside u32), while per-output accessors (cmu, cmx, nf, ephemeral key) return CompactFormatError::{InvalidLength, InvalidValue}; the From impls that *build* compact messages from full bundles take encCiphertext[0..COMPACT_NOTE_SIZE] (zcash_client_backend, src/proto.rs, lines 63–286). The scanner bounds CompactTx.index to u16 (the expect reads "Cannot fit more than 2^{16} transactions in a block"), and CompactBlock.time — uint32 Unix epoch, "if non-zero" — flows into ScannedBlock and thence wallet metadata (src/scanning/compact.rs, lines 314–315, 552–555).

Compact blocks are cached as serialized protos. Crate zcash_client_sqlite offers BlockDb, a compactblocks SQLite table decoded on read (CompactBlock::decode), and FsBlockDb, one file per block plus a BlockMeta row {height, blockhash, time, sapling_outputs_count, orchard_actions_count} that enables range queries without decoding (zcash_client_sqlite, src/chain.rs, lines 30–230).

ZIP 307's own client-operation sketch — append every cmu to a depth-32 incremental Merkle tree, keep a per-note incremental witness, and cache roughly 100 recent tree states because "no full Zcash node will roll back the chain by more than 100 blocks" (ZIP 307, "Creating and updating note witnesses") — describes the pre-shardtree design. The deployed replacement is the position-marked shardtree fed by GetSubtreeRoots (RPC added in lightwallet-protocol v0.3.5; zcash_client_backend, src/sync.rs, lines 80–86), whose mechanics the *Ironwood Guide* constructs in §"Proof of ownership of *some* valid note: the commitment tree".

1.7 Server inclusion semantics and successor surface

Which transactions a compact block contains is itself divergent. ZIP 307 states that "By default, CompactBlocks only contain CompactTxs for transactions that contain Sapling spends or outputs" (ZIP 307, "Block header validation"). Server lightwalletd builds and caches compact representations of *all* transactions, transparent-only ones included, with vin/vout populated (parser/block.go, lines 120–125), and GetBlock serves them unfiltered per the v0.4.1 contract; GetBlockRange filters per request — with empty poolTypes it prunes each transaction to its Sapling and Orchard components and drops transactions left empty (common/common.go, FilterTxPool/filterBlockPool, lines 526–653) — so the default sync stream carries shielded-relevant transactions only, as Zaino's does. The successor Zaino filters out CompactTxs whose compact fields are all empty, and supports per-request pool filtering via PoolTypeFilter and BlockRange.poolTypes (zaino-fetch/src/chain/block.rs, lines 193–215) — a filter added in v0.4.0, where an empty poolTypes means Sapling and Orchard for backward compatibility (service.proto, lines 38–41). Both servers therefore default to shielded-relevant transactions on the sync stream; we flag the deployed defaults against the ZIP in §1.8.

Two lightwalletd RPCs, GetBlockNullifiers and GetBlockRangeNullifiers, strip actions to nullifier-only, drop outputs, vin, and vout, and zero both ChainMetadata tree sizes ("these are not needed (we prefer to save bandwidth)") (frontend/service.go, lines 212–302). Blocks so obtained therefore cannot seed the position ladder of §1.5 — zeroed sizes land in the TreeSizeInvalid/TreeSizeUnknown paths — so the RPCs serve spend-status refresh, not scanning. Both were declared deprecated in the lightwallet-protocol v0.4.0 changelog (2025-12-03); v0.4.1 designated them deprecated in the proto itself (option deprecated = true, service.proto:247,267), in favour of GetBlockRange with poolTypes (service.proto, lines 244–266; CHANGELOG.md, v0.4.0/v0.4.1).

Ironwood. The v0.5.0 fields (ironwoodCommitmentTreeSize = 3, ironwoodActions = 9) exist only in lightwallet-protocol v0.5.0; librustzcash gates the corresponding tree account-

ing on `NetworkUpgrade::Nu6_3`, and lightwalletd — the deployed service — neither parses nor serves Ironwood data (zcash_client_backend, `src/scanning/compact.rs`, lines 687–696; `proto/compact_formats.proto`, lines 17, 74). Per our conventions: the wire fields exist in the canonical proto files and changelog (v0.5.0, 2026-06-30) but in no tagged release — the latest tag is v0.4.1 (`research/upstream-defects.md`, item 3); the pool and its activation are *specified* (ZIP 2005, Proposed; ZIP 258, Draft: Testnet activation 4,134,000, Mainnet scheduled in deployed node software at 3,428,143); nothing deployed serves the compact-format fields.

1.8 Divergences between ZIP-307 and the deployed protocol

Per our conventions, we flag rather than resolve the following.

- *No normative specification of the deployed wire format.* ZIP 307 is Draft, Sapling-only, and its message definitions are stale — old names, old field numbers, no Orchard, no `ChainMetadata`, no `vin`/`vout` — while the lightwallet-protocol README disclaims being a specification and defers to ZIP 307 (`zip-0307.rst`, "Compact Stream Format"; `lightwallet-protocol/README.md`). The deployed format is specified only by tagged proto releases.
- *The grace-period pseudocode bug.* Both "[Canopy onward]" conditions in ZIP 307's compact-decryption procedure test height $<$ CanopyActivationHeight + ZIP212GracePeriod; read literally they reject `0x01` during the grace period and constrain nothing after it. Deployed `zip212_enforcement` accepts both lead bytes during the grace period and requires `0x02` after (`zip-0307.rst`, lines 275–276; zcash_primitives, `src/transaction/components/sapling.rs`, lines 27–40; `research/upstream-defects.md`, item 1, verified 2026-07-15 at zips 69610984).
- *Inclusion semantics.* ZIP 307 says Sapling-relevant transactions only; the deployed default stream includes Orchard data, transparent data is available on request (`BlockRange.poolTypes`), and `GetBlock` returns all pools unconditionally (§1.7).
- *A `protoVersion` that never varies on the wire.* Server lightwalletd never sets it; deployed Zaino serves `0`, byte-identical to an unset proto3 scalar; and v0.5.0 removed it with field number and name reserved (issue `zcash/lightwallet-protocol#25`); librustzcash never reads it (§1.2).
- *Dead header-validation surface.* The proto declares `header` "should always be unset"; ZIP 307's SPV-style header and `finalSaplingRoot` checks are unimplementable against deployed servers, and only `prevHash`/height continuity is enforced client-side (§1.2, §1.5).
- *An advertised but unpopulated `fee` field.* The proto says "present if server can provide"; lightwalletd has a TODO, Zaino hardcodes zero (§1.2).
- *Vendored-revision skew.* lightwalletd vendors v0.4.1, librustzcash main v0.5.0, and Zaino an intermediate snapshot with both the Ironwood fields and `protoVersion`; Zaino's vendored changelog dates v0.5.0 as 2026-07-03 without the `protoVersion` removal, while the canonical changelog dates it 2026-06-30 with the removal — so Zaino's vendored proto still *declares* the field the canonical proto now reserves, though it serves only `0` on the wire (diff of the three `compact_formats.proto` copies; `zaino-proto/lightwallet-protocol/CHANGELOG.md`, lines 40–51).

2 The light-client service

A light wallet holds keys and no chain. The *Ironwood Guide* (§"Transmission and detection of a note: encryption, trial decryption, and synchronisation") establishes what synchronisation

consumes: every compact shielded output on the chain, for trial decryption; every nullifier, for spentness; the note commitment tree's frontier and subtree roots, for witness assembly; and, for the few transactions that prove to be the wallet's own, the full serialised bytes. The deployed layer that delivers these is a single gRPC service, `cash.z.wallet.sdk.rpc.CompactTxStreamer`, spoken between the wallet and an untrusted proxy in front of a full node. This section documents the service as deployed: the authority trail from ZIP 307 through the canonical protobuf definitions (§2.1), the compact data format (§1), the interaction model the ZIP prescribes (§2.3), the twenty RPCs and their semantics (§2.4), the two server implementations — lightwalletd, the deployed proxy (§2.5), and Zaino, its successor (§2.6) — and a register of the points where the implementations and the normative texts disagree (§2.7). The linear costs this architecture accepts — trial decryption of every output, a maintained witness per note — are precisely the costs the Tachyon redesign removes (*Tachyon Guide*, §"What the fold removes").

2.1 Architecture and authority

ZIP 307, "Light Client Protocol for Payment Detection" (Status: Draft), fixes a three-component architecture: a Zcash full node as the root of trust; a *proxy server* that converts chain data into a compact format; and the light client (ZIP 307, § "High-Level Design"). The security model is deliberately narrow: the client obtains *payment detection privacy* against an honest-but-curious proxy, and trusts that proxy to provide a correct view of the best chain; network-level privacy (Tor, Nym) is assumed to be provided separately; spending privacy is out of scope (ZIP 307, § "Security Model"). The chain has carried commitments designed to let a client check that view succinctly since Heartwood (ZIP 221, via `hashBlockCommitments`); no deployed client consumes them (*FlyClient Guide*, §"The deployment gap").

Where the contract lives. The wire contract is *not* in the ZIP. The two protobuf files that define the service — `service.proto` and `compact_formats.proto` — have their canonical home in the `zcash/lightwallet-protocol` repository, which self-describes as the canonical Protobuf definitions but explicitly not a specification: "The Zcash Light Client Protocol is described in ZIP-307" (lightwallet-protocol, `README.md`). The authority trail is therefore circular — the ZIP defers detail to the deployed API, and the API repository defers normative force back to the ZIP — and the circle does not close: ZIP 307's own protobuf sketch (`CompactSpend` and `CompactOutput` messages, a `BlockID` embedded in `CompactBlock`, a `RangeFilter`, a `FullTransaction` return type, a four-RPC `CompactTxStreamer`) differs from the deployed proto (`CompactSaplingSpend`, `CompactSaplingOutput`, `CompactOrchardAction`, flat height and hash fields, `BlockRange`, `RawTransaction`, twenty RPCs), and the ZIP warns that "the details of the API described below may differ from the implementation" (ZIP 307, § "Compact Stream Format", § "Proxy operation"). We treat the canonical proto files, versioned and change-logged, as the deployed contract, and cite the ZIP for architecture and rationale. Both lightwalletd and Zaino vendor the proto files by git subtree (lightwallet-protocol, `README.md`).

Contract version. The canonical protocol is at v0.4.1 (2026-02-20). Version 0.4.0 (2025-12-03) added transparent data to the compact format — `PoolType`, `CompactTxIn`, `TxOut`, `CompactTx.vin/vout`, `BlockRange.poolTypes` — and the `LightdInfo` fields `upgradeName`, `upgradeHeight`, and `lightwalletProtocolVersion`, and declared `GetBlockNullifiers` and `GetBlockRangeNullifiers` deprecated; v0.4.1 designated the pair deprecated in the proto itself (`option deprecated = true`, `service.proto:247,267`) and documented `GetBlock`'s transparent-data behaviour (lightwallet-protocol, `CHANGELOG.md`).

The proto forks. The vendored copies have already diverged from the canonical file, in one direction: fields for the future Ironwood pool. The canonical v0.4.1 has `PoolType` = {`INVALID`, `TRANSPARENT`, `SAPLING`, `ORCHARD`} and `ShieldedProtocol` = {`sapling`,

orchard}. The fork in librustzcash (zcash_client_backend/proto/service.proto) adds IRONWOOD = 4, TreeState.ironwoodTree = 7, and ShieldedProtocol.ironwood = 2. Zaino's fork (zaino-proto) adds IRONWOOD = 4, ironwoodTree = 7, ChainMetadata.ironwoodCommitmentTreeSize = 3, and CompactTx.ironwoodActions = 9, but not ShieldedProtocol.ironwood, and omits the deprecated options on the two nullifier RPCs (diff of the three proto trees; zaino, docs/internal_spec.md, § "Zaino-Proto"). Zaino's documentation states the local copies exist "for development purposes" with a plan to rely on librustzcash's. The "as deployed" wire surface therefore differs by repository; this volume cites the canonical file and flags fork-only fields where they appear.

Remark 2.1 (Sources and pinning)**.** Deployed-behaviour claims in this section were verified against lightwalletd master 61fee32e (2026-06-25), zaino 0e057e22 (2026-07-04), and librustzcash e30517e4 (2026-07-10); Zaino pins zebra-chain 11.0.0 from crates.io (Cargo.lock). File-and-line citations below refer to these revisions.

2.2 The compact format

The proxy's one transformation is compression: it strips from each transaction everything a detecting wallet does not need. Proofs, signatures, and value commitments verify the chain, and the client has delegated chain verification to the proxy; what remains is exactly the trial-decryption input and the nullifier.

Definition 2.2 (Compact Sapling output, ZIP 307)**.** A compact output retains three fields of a Sapling Output description:

$$\texttt{CompactSaplingOutput} = \big(\texttt{cmu}[32],\ \texttt{ephemeralKey}[32],\ \texttt{ciphertext} = \texttt{encCiphertext}[0..52]\big),$$

116 bytes per output against 580 + 32 bytes for the full description with its commitment — an ∼80% reduction (ZIP 307, § "Output Compression"; compact_formats.proto:114--122). The 52-byte ciphertext prefix covers the compact note plaintext — the contents and opening of the note commitment. Truncation discards the AEAD tag; integrity of the decrypted prefix is instead checked by recomputing the note commitment, as the *Ironwood Guide* constructs in §"Transmission and detection of a note: encryption, trial decryption, and synchronisation".

Definition 2.3 (Compact Orchard action, deployed proto)**.** The deployed format extends the same compression to Orchard, one message per Action:

$$\texttt{CompactOrchardAction} = \big(\texttt{nullifier}[32],\ \texttt{cmx}[32],\ \texttt{ephemeralKey}[32],\ \texttt{ciphertext}[52]\big)$$

(compact_formats.proto:124--130). The Orchard message carries its nullifier inline because an Action both spends and creates; on the Sapling side, spend compression is a separate message that reduces the 384-byte Spend description to its 32-byte nullifier — a ∼90% reduction (ZIP 307, § "Spend Compression").

Transparent data. Since v0.4.0 the compact format can carry transparent inputs and outputs (CompactTxIn, TxOut, CompactTx.vin/vout); v0.4.1 fixes the convention that the coinbase transaction's single null-outpoint vin is omitted, and that clients identify the coinbase by testing CompactTx.index = 0 (service.proto:225--237; compact_formats.proto:70--76).

Remark 2.4 (The fee field is dead on the wire)**.** The proto documents CompactTx.fee as "present if server can provide", with the stateless-server formula for the case of no transparent inputs,

$$\texttt{fee} = \texttt{valueBalanceSapling} + \texttt{valueBalanceOrchard} + \sum \texttt{vPubNew} - \sum \texttt{vPubOld} - \sum \texttt{tOut}$$

(compact_formats.proto:59--64); with transparent inputs a stateless server cannot compute the fee at all. Neither deployed server ever sets the field: lightwalletd's parser leaves it at 0

with a "TODO: calculate fees" comment (lightwalletd, `parser/transaction.go:397--432`), and Zaino's conversions pass `fee = None`, encoded 0 (zaino, `zaino-state/src/chain_index/types/db/legacy.rs:1397--1470`). The deployed contract is therefore that the field carries no information, and a client must treat it as absent; we flag the proto's "present if" language as unimplemented on both servers.

2.3 The interaction model of ZIP 307

The ZIP prescribes a four-phase client-server interaction (ZIP 307, § "Client-server interaction"):

(A) *Startup.* The client fetches the latest `BlockID`. A wallet importing a pre-existing seed with no recorded birthday starts at Sapling activation, since no shielded address it can derive predates Sapling (ZIP 307, § "Importing a pre-existing seed").

(B) *First sync.* The client requests `GetBlockRange`(X, Y) and, on any subsequent request, verifies the hash of the overlap block — the last block it already holds — against its cached copy, so that a reorganisation is detected as a hash mismatch.

(C) *Transaction fetching.* Detected notes are completed by fetching full transactions via `GetTransaction`. Each such fetch names a txid to the proxy, so the ZIP mandates cover traffic: clients SHOULD download decoy transactions, or all transactions of a touched block, and MUST warn users of the privacy loss otherwise (ZIP 307, § "Transaction privacy").

(D) *Incremental sync.* Later syncs repeat (B) from the cached tip, re-checking the overlap block.

Two hardening measures described by the ZIP are not deployed. Block-header validation by the client is described as only partially implemented — only `prevHash` linkage is checked (ZIP 307, § "Block header validation"). The *bucketing* privacy enhancement — for bucket size N, start each range request at

$$\text{floor}(X) \;=\; X - (X \bmod N)$$

instead of at the true resume height X, hash-checking and discarding blocks in $[\text{floor}(X), X]$, so that the request leaks only the bucket and absorbs reorganisations — is proposed and not implemented (ZIP 307, § "Block privacy via bucketing"). Both are ZIP-versus-deployed gaps a reader should not assume closed.

The reference client narrows the surface further: the sync loop in zcash_client_backend (`src/sync.rs:1--10,244--381`) exercises four shielded-sync RPCs (below) plus, when built with feature `transparent-inputs`, `GetAddressUtxosStream`, issued per account per cycle by `refresh_utxos` (`src/sync.rs:117--126,511--527`; cf. §5.3):

- `get_latest_block`, in `update_chain_tip`;
- `get_subtree_roots`, in `download_subtree_roots`;
- `get_block_range`, in `download_blocks`;
- `get_tree_state`, in `download_chain_state`.

Transaction enhancement via `GetTransaction` is listed as not yet implemented in that module (`src/sync.rs`). We return to the loop in the next section.

Group	RPC	Sync loop	Notes
Chain	`GetLatestBlock`	✓	tip height and hash
	`GetBlock`		one block, all pools
	`GetBlockRange`	✓	streamed range, pool filter
	`GetBlockNullifiers`		deprecated (v0.4.0; proto option v0.4.1)
	`GetBlockRangeNullifiers`		deprecated (v0.4.0; proto option v0.4.1)
Trees	`GetTreeState`	✓	frontiers at height/hash
	`GetLatestTreeState`		frontiers at the tip
	`GetSubtreeRoots`	✓	depth-16 subtree roots
Transactions	`GetTransaction`		full bytes by txid
	`SendTransaction`		submit raw bytes
Mempool	`GetMempoolTx`		compact mempool contents
	`GetMempoolStream`		full txs until next block
Transparent	`GetTaddressTxids`		deprecated, misnamed
	`GetTaddressTransactions`		
	`GetTaddressBalance`		
	`GetTaddressBalanceStream`		
	`GetAddressUtxos`		
	`GetAddressUtxosStream`		
Metadata	`GetLightdInfo`		server and chain metadata
	`Ping`		testing-only

Table 3: The twenty RPCs of `CompactTxStreamer` (`service.proto:221--326`, lightwallet-protocol v0.4.1). The ✓ column marks the shielded-sync RPCs exercised by the reference sync loop; with feature `transparent-inputs` the loop also issues `GetAddressUtxosStream` per account per cycle (zcash_client_backend, `src/sync.rs`).

2.4 The RPC surface

The service defines twenty RPCs (`service.proto:221--326`); Table 3 groups them. Three are deprecated, one is testing-only, and six serve the transparent layer, outside this volume's shielded-sync scope. The four marked ✓ are the ones the reference sync loop consumes for shielded sync (§2.3).

Remark 2.5 (Hash orders)**.** Two byte orders cross the interface. The byte-typed fields `BlockID.hash` and `CompactBlock.hash` carry raw bytes in little-endian (protocol) order; lightwalletd reverses the big-endian hex it receives from the node's RPCs (lightwalletd, `frontend/service.go:69--90`). Field `TreeState.hash`, by contrast, is big-endian display hex *text* (`service.proto:176--184,307--312`). Requests follow the same split: `TxFilter` takes protocol-order txid bytes that the server reverses into display hex for `getrawtransaction` (`frontend/service.go:403--442`), and the mempool exclude entries are protocol-order byte suffixes the server reverses into display-order prefixes (`frontend/service.go:600--747`). Every byte-typed hash on this interface is protocol order; every string-typed hash is display order.

Chain tip. The RPC `GetLatestBlock(ChainSpec)` returns a `BlockID` — height and hash of the best chain tip; lightwalletd proxies `getblockchaininfo` and returns the hash as little-endian raw bytes (lightwalletd, `frontend/service.go:69--90`). This is the chain tip the wallet's confirmation accounting consumes (*Wallet Guide*, §"Confirmations, trust, and spendability").

Single block. The RPC `GetBlock(BlockID)` returns one `CompactBlock`, and per the v0.4.1 contract includes transaction data for *all* value pools, transparent `vin`/`vout` included — unlike `GetBlockRange`, which defaults to shielded-only (`service.proto:225--237`). The proxy lightwalletd serves the request from its block cache first and falls back to the node on a miss; a height above the tip returns gRPC `OutOfRange` ("block %d is newer than the latest block"), a lookup by hash returns `InvalidArgument` ("not yet implemented", referencing PR #309), and height 0 with no hash is `InvalidArgument` (lightwalletd, `common/common.go:495--624`, `frontend/service.go:166--189`). Zaino accepts lookup by hash — hash takes precedence and is resolved to a height via its chain index — and builds the block with all pools included, matching the v0.4.1 contract; the proto makes hash support optional, so both behaviours are conformant, but they diverge (zaino, `zaino-state/src/backends/state.rs:1954--2058`; `zaino-proto/src/proto/utils.rs:310--323`).

Block ranges. The RPC `GetBlockRange(BlockRange)` streams consecutive `CompactBlocks` over the closed interval $[\min(s,e), \max(s,e)]$, in increasing height order iff $s \leq e$ and decreasing otherwise — lightwalletd computes $j = high - (i - low)$, Zaino tests $s \leq e$ (`service.proto:250--255`; lightwalletd, `common/common.go:574--624`; zaino, `zaino-state/src/chain_index.rs:1774--1802`). An empty `poolTypes` field selects the legacy behaviour: prune each block to shielded (Sapling and Orchard) data only; clients MUST verify server capability before setting `poolTypes` non-empty (`service.proto:28--42`). Filtering prunes transaction components and then drops empty transactions, but a block whose transactions were all filtered is still returned, with empty `vtx` (lightwalletd, `common/common.go:574--624`). Zaino validates the request up front — start and end must be present and fit in `u32`; `POOL_TYPE_INVALID` or unknown pool values are `InvalidArgument` — streams through an mpsc channel of the configured `channel_size`, caps an ascending range at the tip and appends a trailing `OutOfRange` error after the valid blocks, and bounds production by a hard timeout of 4× the service timeout (zaino, `zaino-state/src/backends/state.rs:550--683`; `zaino-proto/src/proto/utils.rs:53--152`).

Nullifier variants. The RPCs `GetBlockNullifiers` and `GetBlockRangeNullifiers` — both deprecated (v0.4.0 changelog; proto option v0.4.1) in favour of `GetBlockRange` with

poolTypes — return CompactBlocks reduced to shielded nullifier data: lightwalletd strips Sapling outputs and transparent vin/vout, reduces each Orchard action to its nullifier, and zeroes both commitment-tree sizes; GetBlockRangeNullifiers MUST ignore any TRANSPARENT member of poolTypes, and lightwalletd filters it out (service.proto:239--268; lightwalletd, frontend/service.go:191--225,259--305). Zaino additionally accepts these by hash, as for GetBlock (zaino, zaino-state/src/backends/fetch.rs:929--1041).

Tree state. The RPC GetTreeState(BlockID) returns the note commitment tree state for a block specified by height or hash: network name, height, block hash as a hex string, block time, and the saplingTree and orchardTree frontiers as hex (service.proto:176--184). In lightwalletd the reply derives from the node's z_gettreestate RPC, looping along Sapling.SkipHash when a reply carries no FinalState; GetLatestTreeState calls the same path at getblockchaininfo's tip height (lightwalletd, frontend/service.go:307--401). The frontier is what seeds the wallet's shard tree below its birthday, so that witness assembly never replays the chain's prefix (*Ironwood Guide*, §"Proof of ownership of *some* valid note: the commitment tree"). Zaino resolves hash-or-height, reports the BIP70 network name, and returns a seventh field ironwoodTree (hex, empty when absent) that exists only in Zaino's and librustzcash's proto forks (§2.1); GetLatestTreeState reuses GetTreeState at the current chain height (zaino, zaino-state/src/backends/state.rs:2516--2576; zaino-proto/proto/service.proto:183). One stale cross-reference: the proto comment for GetTreeState cites "section 3.7 of the Zcash protocol specification", while the current protocol.tex numbers note commitment trees as § 3.8.

Subtree roots. Shielded outputs of a pool are numbered from genesis, and each consecutive group of $2^{16} = 65536$ note commitments forms one depth-16 subtree of the pool's commitment tree; the stream element is

$$\texttt{SubtreeRoot}_i = \big(\text{Merkle root of subtree } i,\ \text{hash and height of the block containing output } (i+1)\cdot 2^{16}-1\big),$$

so that, for example, Sapling output 65535 — the last of subtree 0 — falls in block 558822 (service.proto:191--200; lightwalletd, common/common.go:178--219). The RPC GetSubtreeRoots(GetSubtreeRootsArg) streams these for the chosen shielded protocol from startIndex, with maxEntries = 0 meaning all. The lightwalletd handler calls z_getsubtreesbyindex and then, per subtree, fetches the completing block via GetBlock at its end height to obtain the hash, reversed to big-endian display-order bytes, the opposite convention from CompactBlock.hash (Remark 4.6; lightwalletd, frontend/service.go:832--907). These roots are exactly the retained internal nodes from which the wallet's shard tree assembles authentication paths without replaying every commitment (*Orchard Guide*, §"Proof of ownership of *some* valid note: the commitment tree"). Zaino converts startIndex and maxEntries to u16 and returns InvalidArgument when either exceeds u16 range, where lightwalletd forwards the full uint32 — bounded by design, since a depth-32 tree has at most 2^{16} depth-16 subtrees, but the wire contracts differ; it fills the completing block data via z_get_block per subtree, under a 4× service-timeout deadline. One Zebra-specific caveat: during Zebra's initial subtree-index rebuild the underlying RPC returns an empty list for not-yet-rebuilt indices, where zcashd rebuilds before serving (zaino, zaino-state/src/indexer.rs:429--452,754--912).

Full transactions. The RPC GetTransaction(TxFilter) returns the full serialised transaction. Only lookup by 32-byte txid is supported; a block-plus-index lookup returns InvalidArgument ("specify a txid, not a blockhash+num"). The lightwalletd handler converts the little-endian txid bytes to big-endian hex, calls getrawtransaction with verbose=1, and maps RPC failure to gRPC NotFound (service.proto:44--51,270--271; lightwalletd,

`frontend/service.go:403--442`). The privacy cost of this call — it names a txid of interest to the proxy — is the reason for the ZIP's decoy mandate (§2.3).

Submission. The RPC `SendTransaction(RawTransaction)` submits via the node's `sendrawtransaction`. Success returns `SendResponse{errorCode=0, errorMessage=`txid hex`}`; a node rejection is parsed from the "code: message" error string into a non-zero `errorCode` plus message — node-level rejection travels in-band, not as a gRPC error, so a transport-level success is not an acceptance (`service.proto:83--89`; lightwalletd, `frontend/service.go:454--509`). Wallet-side obligations around this call — store-then-broadcast ordering, rebroadcast, expiry — are the *Wallet Guide*'s subject (§"Store, then broadcast").

Remark 2.6 (The `RawTransaction.height` sentinels)**.** An original protobuf-definition error typed the field `uint64` rather than `int64`, forcing sentinel semantics (`service.proto:53--81`; lightwalletd, `common/common.go:626--661`):

$$\texttt{height} = \begin{cases} 0 & \text{transaction in the mempool,} \\ 2^{64}-1 \ (\texttt{0xffffffffffffffff}) & \text{mined on a non-main-chain fork (zcashd's } -1), \\ h & \text{mined on the main chain at height } h. \end{cases}$$

The proto's own comments contradict each other: the message-level comment says the field carries "the latest block height" when returned by `GetMempoolStream`, the field-level comment specifies the sentinel mapping above, and a FIXME cites librustzcash issue #1484. The deployed servers realise both readings — lightwalletd streams 0, Zaino streams the current best-tip height (§2.7). We flag the mempool-height contract as unresolved (`research/upstream-defects.md`, which tracks this as the fourth known defect, already filed as zcash/librustzcash#1484).

Mempool contents. The RPC `GetMempoolTx(GetMempoolTxRequest)` streams `CompactTx` for the current mempool contents, possibly a few seconds stale. The request may carry `exclude_txid_suffixes`: txid byte-suffixes of any length up to 32 bytes, in client-side protocol order; the server reverses and hex-encodes each into a display-order prefix, and a mempool transaction is excluded iff its txid matches a prefix that matches *exactly one* mempool txid — when two or more txids match, none of the matching transactions are excluded, and unmatched entries are ignored (`service.proto:157--174,289--301`; lightwalletd, `frontend/service.go:600--747`). Field 2 of the request is reserved for a future `exclude_txid_prefixes`. An empty `poolTypes` defaults to Sapling-plus-Orchard pruning; `POOL_TYPE_INVALID` is rejected. The lightwalletd implementation backs the RPC with a process-global cache — `mempoolMap` from txid to `CompactTx`, plus `mempoolList` — refreshed at most every 2 s from `getrawmempool` plus per-transaction `getrawtransaction verbose=0`, reusing already-fetched entries across refreshes; mempool `CompactTx`s are built with height 0 and no fee (lightwalletd, `frontend/service.go:589--708`). Zaino enforces the same 32-byte suffix bound and the same exactly-one-match exclusion rule, parses transactions from stored serialised bytes, and prunes by the requested pool types before streaming (zaino, `zaino-state/src/backends/state.rs:2302--2429`, `chain_index/mempool.rs:381--417`).

Mempool stream. The RPC `GetMempoolStream(Empty)` streams full `RawTransaction`s of the current mempool and keeps the stream open, closing it when a new block is mined (`service.proto:303--305`). The lightwalletd implementation is process-global state — `g_txList` in arrival order, a `g_txidSeen` set, `g_lastBlockChainInfo` — polled at most every 2 s via `getblockchaininfo` and `getrawmempool`; the first stream to observe a tip-hash change resets the shared state and all open streams terminate; each per-client loop sleeps 200 ms between sends; transactions mined since listing (height $\neq 0$) are skipped; streamed transactions carry `height` $= 0$ (lightwalletd, `common/mempool.go:14--152`, `frontend/service.go:581--587`).

Zaino streams `height` = current best-tip height, closes the stream when its mempool status flips to `Syncing` on a tip change, and additionally imposes a hard 6× service-timeout deadline — 180 s at defaults — where lightwalletd keeps the stream open until the next block regardless of duration; a long block interval can therefore end a Zaino stream with a deadline error where the proto's documented close-on-new-block semantics would hold (zaino, `zaino-state/src/backends/state.rs:2433--2510`, `chain_index/mempool.rs:419--480`).

Server metadata. The RPC `GetLightdInfo(Empty)` returns server metadata plus chain state assembled from `getinfo` and `getblockchaininfo`: version, vendor ("ECC LightWalletD"), `taddrSupport` true, chain name, the Sapling activation height — looked up via the Sapling consensus branch ID `0x76b809bb` in the upgrades map — the chain tip's consensus branch ID, block height, build and git fields, `estimatedHeight` (less than the tip while the node itself syncs), donation address, and the next pending upgrade's name and height (`service.proto:97--118`; lightwalletd, `common/common.go:261--319`). The proxy does *not* populate the proto's `lightwalletProtocolVersion` field 18. Zaino reports vendor "ZingoLabs ZainoD", hard-codes `lightwallet_protocol_version` = "v0.5.0" — a version with no tagged release in the canonical repository, the latest tag being v0.4.1, though an untagged v0.5.0 changelog section (2026-06-30) exists (`research/upstream-defects.md`, item 3; `zaino-state/src/backends/fetch.rs:2097`, `state.rs:2776`) — defaults the Sapling activation height to 1 on regtest, and carries the connected Zebra node's build information in the `zcashdBuild`/`zcashdSubversion` fields (zaino, `zaino-state/src/backends/state.rs:2724--2793`). Both sides are flagged in §2.7.

Ping. The RPC `Ping` is testing-only; lightwalletd disables it unless started with the flag `--ping-very-insecure`, because the call lets a client create arbitrarily many concurrent server threads — a resource-exhaustion risk (`service.proto:324--325`; lightwalletd, `frontend/service.go:920--936`). Zaino returns gRPC `Unimplemented` (zaino, `zaino-state/src/backends/state.rs:2782--2792`).

Transparent-address RPCs. The six remaining RPCs (`GetTaddressTxids` — deprecated and misnamed, it returns transactions — `GetTaddressTransactions`, `GetTaddressBalance`, `GetTaddressBalanceStream`, `GetAddressUtxos`, `GetAddressUtxosStream`) proxy the node's insight-explorer-derived RPCs `getaddresstxids`, `getaddressbalance`, and `getaddressutxos`; they serve the transparent layer and fall outside this volume's shielded-sync scope. For these, lightwalletd validates t-addresses against the regex `\At[a-zA-Z0-9]{34}\z` and caps each `GetTaddressTransactions` inner fetch loop with a 30-second context timeout (lightwalletd, `frontend/service.go:55--164,511--579,749--830`). Zaino caps the request's address list at `UTXO_MAX_ADDRESSES` = 1000 (`InvalidArgument` beyond) — a server-side fan-out bound lightwalletd lacks — and honours `start_height` and `max_entries` (0 meaning unlimited) as response-size bounds (zaino, `zaino-state/src/backends.rs:29--45`, `backends/state.rs:2591--2719`).

2.5 lightwalletd: the deployed proxy

The deployed server is a Go proxy: each gRPC handler in `frontend/service.go` translates its request into node JSON-RPC calls through the indirect function `common.RawRequest`. It supports both zcashd and zebrad backends, detecting the backend from the node's subversion string (`/Zebra:` versus `/MagicBean:`); a zcashd backend must have the `lightwalletd` or `insightexplorer` experimental feature enabled or lightwalletd exits; the default assumed node is zebrad (lightwalletd, `cmd/root.go:200--263`, `common/common.go:27--35,61--64`).

Ingestion. A single `BlockIngestor` goroutine polls `getbestblockhash` every 2 s; when the tip differs it fetches the next block via *two* `getblock` calls — first `verbose=1` for the txids and hash, a workaround because the built-in parser miscomputes v5 transaction IDs (issue #392),

then the raw block by that hash, so a reorganisation between the calls cannot mix two blocks. A `prevHash` mismatch against the cache tip is treated as a reorganisation and exactly one block is dropped (`Reorg(height-1)`) per iteration; a `getblock` failure retries after 8 s (lightwalletd, `common/common.go:321--493`). The 120 s sleep (`common/common.go:483--489`) runs only when the cache is empty (height = `GetFirstHeight()`) and the node returned no block at that height; with the cache now starting at height 0 (`cmd/root.go:296--299`) the branch is reachable only against a node that cannot yet serve genesis, and its log line ("Waiting for … Sapling activation height") is stale wording, not stale behaviour.

The block cache. The only persistent state lightwalletd maintains is the compact-block cache: two append-only files per chain, `<data-dir>/db/<chain>/lengths` and `.../blocks`. With $d =$ protobuf(`CompactBlock`) the entry for height h is

$$\mathrm{FNV1a}_{64}\big(\mathrm{LE}_{64}(h) \,\|\, d\big) \,\|\, d \quad \text{in } \texttt{blocks}, \qquad \mathrm{LE}_{32}\big(|d|\big) \quad \text{in } \texttt{lengths},$$

with the offset table reconstructed on startup and valid lengths constrained to $74 \le |d| \le 4{,}000{,}000$ bytes; corruption — a bad checksum, an impossible length, a partial entry — triggers automatic cache clearing and redownload (lightwalletd, `common/cache.go:21--230`). The cache now starts at height 0, previously Sapling activation, because compact blocks carry transparent data (lightwalletd, `cmd/root.go:265--300`). All other responses are proxied per-request; the mempool caches of §2.4 are in-memory only; no per-client state is kept.

Operational defaults. gRPC binds 127.0.0.1:9067 and HTTP 127.0.0.1:9068 (Prometheus `/metrics`); TLS is required — the server exits if the certificate or key is unloadable — unless `--no-tls-very-insecure` (plaintext) or `--gen-cert-very-insecure` (self-signed); the data directory defaults to `/var/lib/lightwalletd`; `--nocache` disables the disk cache; `--sync-from-height` and `--redownload` control cache rebuild; the darkside mock mode (`--darkside-very-insecure`) auto-terminates after 30 minutes (lightwalletd, `cmd/root.go:139--180,375--424,498--499`).

2.6 Zaino: the successor indexer

Zaino is the successor indexer, in Rust: `zainod` is the daemon, `zaino-serve` implements `CompactTxStreamer` over tonic, `zaino-state` holds the chain index and backends, `zaino-fetch` is the JSON-RPC client, and `zaino-proto` vendors the protos. The stated motivations are the zcashd deprecation, a Rust stack alongside librustzcash and Zebra, and a clean indexer/validator trust boundary: indexing functionality moves out of the validator into Zaino, with Zebra exposing its `ReadStateService` for direct read access (zaino, `README.md`, § "Motivations", § "Project Structure"; `docs/internal_spec.md`).

Backends. Configuration `backend = 'fetch' | 'state'` selects one of two chain-fetch backends. Backend `StateService` is backed by Zebra's `ReadStateService` plus a `TrustedChainSync` (`init_read_state_with_syncer`): it reads Zebra's state store directly, and at startup blocks until the syncer's tip height *and* hash match the validator's JSON-RPC tip — hash compared because height alone is ambiguous during a reorganisation. Backend `FetchService` is backed by the node's JSON-RPC interface (zcashd back-compatibility, or a remote Zebra) and is marked "`#[deprecated]`: Will be eventually replaced by `BlockchainSource`". Both feed the same `NodeBackedChainIndex` (zaino, `zaino-state/src/backends/state.rs:1,117--260`, `backends/fetch.rs:1,83--100`). The shipped example configuration nonetheless selects `backend = 'fetch'` — the deprecated backend — and the repository documentation does not say which backend is the recommended production deployment (zaino, `docs/example_configs/zainod.toml`); we flag the question as open.

The chain index. Server-side state is the `ChainIndex`, with three components (zaino, `docs/internal_spec.md`, § "Zaino-State"; `zaino-state/src/chain_index.rs:1--100`). A `Mempool` — a dashmap broadcast map keyed by txid, whose sync task polls `get_best_block_hash` and on a tip change clears the map and notifies subscribers (`chain_index/mempool.rs`). A `NonFinalisedState` — an in-memory `ArcSwap` snapshot of the top blocks of all chains. A `FinalisedState` — all blocks below the seam — served either by a versioned LMDB-backed persistent database (v1) or, with `ephemeral_finalised_state`, by a stateless passthrough to the validator; large syncs and version migrations run in the background while serving continues (`chain_index/finalised_state.rs:1--80`). The seam is

$$\text{floor}(t) \;=\; t - D_{\text{op}} \quad \text{(saturating at genesis)},$$

where t is the tip height and $D_{\text{op}} = \texttt{OPERATIONAL\_NFS\_DEPTH} = \texttt{MAX\_BLOCK\_REORG\_HEIGHT} + 1 = 1001$ — Zebra's reorganisation limit is local node policy, not consensus; the finalised database serves heights $\leq \text{floor}(t)$, the in-memory state serves heights above it, and in-memory retention is hard-capped at $\texttt{MAX\_NFS\_DEPTH} = D_{\text{op}} + 10 = 1011$ blocks below the tip (zaino, `zaino-common/src/consensus.rs:16--17`, `chain_index/non_finalised_state.rs:23--58`; zebra, `zebra-chain/src/parameters/constants.rs:30`).

Serving. Each gRPC handler delegates to a `LightWalletIndexer` trait implemented by both backend subscribers; streaming responses are tokio mpsc channels wrapped in typed streams (zaino, `zaino-serve/src/rpc/grpc/service.rs:159--320`). The gRPC server may bind a public address only with TLS configured — rejected at startup otherwise — and the default listen address is 127.0.0.1:8137; Zaino additionally serves a zcashd-style JSON-RPC server, unencrypted and restricted to loopback or private bind addresses by default (zaino, `README.md`, § "Server network exposure"; `zainod/src/config.rs:239`). Service defaults: timeout 30 s, `channel_size` 32; stream deadlines are 4× the timeout (120 s), and 6× for `GetMempoolStream` (180 s) (zaino, `zaino-common/src/config/service.rs:8--20`). Until the non-finalised snapshot exists, `GetLatestBlock` and `GetMempoolStream` return an `UnavailableNotSyncedEnough` error where lightwalletd would proxy the node's answer immediately; a code TODO says this "probably shouldn't be an error", so the startup behaviour may change (zaino, `zaino-state/src/backends/state.rs:1940--1951`).

Compatibility. Zaino's integration test suite includes a `client_rpcs` partition that compares its LightWallet gRPC outputs against lightwalletd's, and the project has committed to serving all Zcash JSON-RPC services required by non-validator clients, superseding zcashd in that role (zaino, `docs/internal_spec.md`, § "Zaino-Testutils and Integration-Tests"; `docs/rpc_api.md`).

2.7 Divergence register

Table 4 collects the contract-level points at which the two deployed servers differ from each other or from the normative texts; each row is developed, with citations, in the paragraph of §2.4–§2.6 that covers the RPC. The ZIP-versus-deployment gaps — the message sketch, partial header validation, unimplemented bucketing — are flagged in §2.1 and §2.3; the proto forks in §2.1.

Two rows deserve emphasis. The mempool-height row is a live specification bug: the proto's two comments cannot both be satisfied, the deployed servers realise one reading each, and the FIXME in the canonical file (librustzcash issue #1484) records the conflict without resolving it; a client must therefore treat the height of a streamed mempool transaction as carrying no portable information beyond "not yet mined". The fee row is a silent one: nothing on the wire distinguishes "fee is zero" from "fee not provided", so the proto's conditional language is, in deployed terms, a constant.

Contract point	lightwalletd	Zaino
`GetBlock(Nullifiers)` by hash (optional in proto)	`InvalidArgument`, "not yet implemented" (PR #309)	supported; hash takes precedence
Stream deadlines (proto: `GetMempoolStream` closes on new block)	none; open until next block	hard 4×/6× timeout (120 s/180 s at defaults)
`RawTransaction.height` in `GetMempoolStream` (proto comments contradict; issue #1484)	0	current best-tip height
`GetSubtreeRoots` argument range (proto: `uint32`)	forwards full range to node	`InvalidArgument` above `u16::MAX`
`CompactTx.fee` (proto: "present if server can provide")	always 0 (parser TODO)	always 0 (`fee = None`)
`lightwalletProtocolVersion` (`LightdInfo` field 18, v0.4.0)	never populated	hard-coded "v0.5.0", no tagged canonical release (latest tag v0.4.1)
Behaviour before server is synced	proxies the node immediately	`UnavailableNotSyncedEnough` on `GetLatestBlock`, `GetMempoolStream`
`Ping`	disabled unless `--ping-very-insecure`	`Unimplemented`
Transparent-address fan-out	unbounded address lists	capped at 1000 addresses

Table 4: Deployed divergences across the `CompactTxStreamer` contract. Citations for each row appear in the covering paragraphs of §2.4–§2.6.

3 The scanning pipeline

A light wallet holds no chain. Everything it knows — which notes it owns, which of them are spent, and where each sits in the note commitment tree — it reconstructs by scanning a stream of *compact blocks* supplied by an indexing server. This section develops the deployed reconstruction pipeline of librustzcash's zcash_client_backend and zcash_client_sqlite crates: the compact input format (§1), the account birthday that bounds the scan (§3.2), the priority queue that orders it (§3.3), chain-tip tracking (§3.4), the per-range scanning machinery (§3.5), persistence (§3.6), reorganisation recovery (§3.7), and the reference loop that assembles the parts (§3.8).

The cost shape of the pipeline deserves stating first. For every block in a suggested range, every compact output and action of every transaction is trial-decrypted against every prepared incoming viewing key of every tracked account — two ZIP-32 scopes per pool per account (§3.5) — so synchronisation work scales with global chain traffic multiplied by tracked keys, and is independent of the wallet's own activity; restore-from-seed replays the whole computation from the birthday (zcash_client_backend, `src/data_api/chain.rs:598--699`). This is precisely the deployed cost centre that the *Tachyon Guide*'s proof-carrying wallet is designed to remove (§"The proof-carrying wallet", cost-centre paragraph "Trial decryption of the chain").

3.1 The compact stream

ZIP 307 (Status: Draft) defines the light-client protocol for payment detection. Its *compact output* retains, of a full shielded output, the 32-byte note commitment, the 32-byte ephemeral key, and the first 52 bytes of the note ciphertext:

$$\underbrace{32}_{\text{commitment}} + \underbrace{32}_{\mathsf{epk}} + \underbrace{52}_{\text{ciphertext prefix}} = 116 \text{ bytes},$$

an ∼80% reduction against the full 580-byte ciphertext plus 32-byte ephemeral key; its *compact spend* is the 32-byte nullifier alone, an ∼90% reduction against the 384-byte Sapling Spend description (ZIP 307 §§"Output Compression", "Spend Compression"). Truncating at 52 bytes discards the AEAD tag, so a compact decryption authenticates differently from a full one: the wallet recomputes the note commitment from the decrypted compact plaintext and compares it with the transmitted commitment, trusting the transaction assembler for integrity (ZIP 307 §"Compact Stream Format"). The cryptography of trial decryption — key agreement, KDF, the 52-byte compact note plaintext, the commitment recomputation — is the *Ironwood Guide*'s material (§"Transmission and detection of a note: encryption, trial decryption, and synchronisation"); this volume treats it as a black box mapping a (compact output, incoming viewing key) pair to a note or $\perp$.

The deployed wire format. The format in production has evolved well beyond ZIP 307's message definitions. A block is

```
CompactBlock { reserved 1; height=2; hash=3; prevHash=4; time=5;
               header=6; vtx=7; chainMetadata=8 }
```

where field 1 (`protoVersion`) has been removed and `ChainMetadata` carries the running per-pool note commitment tree sizes `saplingCommitmentTreeSize=1`, `orchardCommitmentTreeSize=2`, and `ironwoodCommitmentTreeSize=3` — the field on which position tracking (§3.5) depends. Each `CompactTx` carries its index, txid, fee, Sapling `spends` and `outputs`, Orchard `actions`, Ironwood actions (`ironwoodActions`, field 9, reusing the `CompactOrchardAction` message), and transparent `vin`/`vout` (zcash_client_backend, `lightwallet-protocol/walletrpc/compact_formats.proto`, vendored at librustzcash `e30517e4`).

Remark 3.1 (Normative anchoring)**.** ZIP 307's message layout — a `CompactBlock` with an embedded `BlockID`, messages `CompactSpend` and `CompactOutput` — does not match the deployed format, and the ZIP remains Draft and Sapling-only. No ZIP specifies the deployed compact format or service: `ChainMetadata` tree sizes, `CompactOrchardAction`, `ironwoodActions`, `GetTreeState`, and `GetSubtreeRoots` exist only in the `zcash/lightwallet-protocol` proto files, which we therefore cite as the specification of record. Ironwood's compact-format extensions are likewise absent from ZIP 2005, which covers quantum recoverability, not light-client synchronisation (`compact_formats.proto`; ZIP 307; ZIP 2005).

The fee field. The `fee` of a `CompactTx` is calculable by a stateless server only for transactions without transparent inputs, as

$$\begin{aligned}\mathsf{fee} = \mathsf{valueBalanceSapling} + \mathsf{valueBalanceOrchard} + \mathsf{valueBalanceIronwood} \\ + \textstyle\sum \mathsf{vPubNew} - \sum \mathsf{vPubOld} - \sum \mathsf{tOut};\end{aligned}$$

with transparent inputs the fee depends on prior-transaction outputs a stateless server cannot provide (`compact_formats.proto:63--69`).

Service surface. The reference synchroniser drives four RPCs of the `CompactTxStreamer` service: `GetLatestBlock(ChainSpec)` → `BlockID`; `GetBlockRange(BlockRange)` → `stream CompactBlock`, addressed by heights only, with an optional `poolTypes` pruning filter that clients MUST verify the server supports; `GetTreeState(BlockID)` → `TreeState`, returning network, height, hash, time, and the hex-serialised Sapling, Orchard, and Ironwood commitment trees; and `GetSubtreeRoots(GetSubtreeRootsArg)` → `stream SubtreeRoot`, returning complete-subtree roots with their completing block hash and height, selected by `enum ShieldedProtocol { sapling=0; orchard=1; ironwood=2 }` (`lightwallet-protocol/walletrpc/service.proto:30--231`).

Remark 3.2 (Deployment divergence at NU6.3)**.** The deployed lightwalletd tree (commit `61fee32e`, 2026-06-25) contains no Ironwood support: its `walletrpc` protos and Go code lack `ironwoodActions`, `ironwoodCommitmentTreeSize`, `ironwoodTree`, and `ShieldedProtocol.ironwood`. The protos vendored by librustzcash (commit `e30517e4`) and Zaino (commit `0e057e22`) define all four, and the reference synchroniser unconditionally requests Ironwood subtree roots (zcash_client_backend, `src/sync.rs:279--296`). An NU6.3-aware wallet therefore requires Zaino or an updated lightwalletd.

3.2 The account birthday

Definition 3.3 (Account birthday)**.** Type `AccountBirthday` pairs a `prior_chain_state: ChainState` with a `recover_until: Option<BlockHeight>`. The *birthday height* is

$$\mathsf{birthday} := \mathsf{priorChainState.height} + 1,$$

the height of the first block to be scanned (zcash_client_backend, `src/data_api.rs:3013--3106`). A `ChainState` holds a block height, a block hash, and the final note commitment tree frontier of each pool: Sapling (depth `sapling::NOTE_COMMITMENT_TREE_DEPTH` = 32), Orchard, and Ironwood — the Ironwood tree is Orchard-shaped (`MerkleHashOrchard`, the same depth) but the pool is distinct and keeps its own tree (zcash_client_backend, `src/data_api/chain.rs:504--596`).

Constructor `AccountBirthday::from_treestate` builds the birthday from a lightwalletd `TreeState` for the last block *prior* to the birthday height. The conversion (`TreeState::to_chain_state`) hex-decodes the block hash — reversing the bytes, since zcashd renders block hashes byte-reversed in hex — parses the height, and converts the hex-serialised Sapling, Orchard, and Ironwood trees into per-pool frontiers; an empty tree field yields an empty tree (zcash_client_backend, `src/data_api.rs:3055--3073`; `src/proto.rs:367--463`).

Choosing a birthday. For a new wallet the crate documentation directs constructing the birthday from the treestate at tip − 100: placing the birthday below the pruning depth guarantees that no reorganisation can mine a transaction intended for the wallet *below* its birthday and so cause funds to be missed (zcash_client_backend, `src/data_api.rs:3199--3211`). For a restored wallet the birthday is historical, and the `recover_until` height marks the chain tip at restoration time: the wallet is *in recovery* until no unscanned ranges remain between the birthday height and `recover_until` (exclusive). The field exists to avoid confusing shifts of balance and spendability while restore-from-seed is in flight — spendability policy itself is the *Wallet Guide*'s `ConfirmationsPolicy` (§"Transaction construction"; §"Broadcast, expiry, and the transaction lifecycle"), which this volume does not restate. Progress reporting follows the same split: type `Progress` carries two scanned-notes/notes-on-chain ratios, one over the recovery window (birthday to recovery height) and one over the scan window (recovery height to tip); either denominator may be zero (zcash_client_backend, `src/data_api.rs:3042--3047,788--831`).

The wallet birthday. The birthday of the *wallet* is the minimum birthday over its accounts — literally `SELECT MIN(birthday_height) FROM accounts` in the SQLite backend — and adding an account whose birthday lies below the current chain tip triggers a rescan of all blocks at and above that height (zcash_client_sqlite, `src/wallet.rs:2967--2978`; zcash_client_backend, `src/data_api.rs:3153--3156`). Function `add_account` persists the birthday height together with the Sapling and Orchard frontier tree sizes, calls `rewind_to_chain_state` targeting the birthday's prior chain state — installing a `Historic`-priority rescan range above birthday − 1 up to the wallet's known chain tip while preserving higher-priority queue entries — and finally inserts an `Ignored`-priority range covering [saplingActivation, birthday) (zcash_client_sqlite, `src/wallet.rs:432--628,4056--4224`).

3.3 The scan queue

The wallet does not scan the chain linearly; it maintains a priority queue of height ranges and always works on the most urgent.

Definition 3.4 (Scan range)**.** A `ScanRange` is a half-open range [start, end) of block heights paired with a `ScanPriority`, with helpers `truncate_start`, `truncate_end`, and `split_at` (zcash_client_backend, `src/data_api/scanning.rs:31--125`). The SQLite backend stores the queue in table `scan_queue` as rows `(block_range_start, block_range_end, priority)` with both bounds `UNIQUE` and a `CHECK` constraint start < end (zcash_client_sqlite, `src/wallet/db.rs:1017--1028`).

Priority	Code	Meaning (doc comment)
`Ignored`	0	Lowest priority; never scanned.
`Scanned`	10	Already scanned; not re-scanned.
`Historic`	20	Advance the fully-scanned height.
`OpenAdjacent`	30	Ranges adjacent to heights at which the user opened the wallet (vestigial; Remark 3.5).
`FoundNote`	40	Complete tree shards adjacent to found notes.
`ChainTip`	50	Complete the latest shard.
`Verify`	60	Previously scanned range that must be re-verified as still on the main chain (highest).

Table 5: The `ScanPriority` ladder, ascending, with its SQLite codes (zcash_client_backend, `src/data_api/scanning.rs:11--29`; zcash_client_sqlite, `src/wallet/scanning.rs:34--45`).

Remark 3.5 (Six live priorities plus one vestigial)**.** We present the ladder as six live priorities and one vestige. Priority `OpenAdjacent` is declared and mapped to SQL code 30, but no code path in the current librustzcash tree ever assigns it: a search at commit `e30517e4` finds only the enum declaration, the code mapping, and two doc-comment mentions (zcash_client_backend, `src/data_api/scanning.rs:20--21`). It still participates in the ordering, so we retain it in Table 5.

Suggesting work. Method `WalletRead::suggest_scan_ranges` returns queue entries in descending priority order. Its documented contract has two clauses: `Verify` ranges must be scanned before all others, to avoid continuity errors under reorganisation; and the compact-block source must supply note commitment tree size metadata (zcash_client_backend, `src/data_api.rs:1995--2010`). The SQLite implementation orders by `priority DESC, block_range_end DESC` — nearest the tip first within a priority — and filters to priorities at or above `Historic` (zcash_client_sqlite, `src/wallet/scanning.rs:61--90`; zcash_client_sqlite, `src/lib.rs:958--960`).

Remark 3.6 (Contract, not type)**.** The guarantee that a `Verify` range arrives first is a documented contract of the trait realised by the SQLite `ORDER BY`; nothing in the type system enforces it, and a third-party `WalletRead` implementation could violate it. We state it as a trait obligation and cite the one deployed implementation that discharges it (zcash_client_backend, `src/data_api.rs:1995--2010`; zcash_client_sqlite, `src/wallet/scanning.rs:61--90`).

Queue maintenance. Function `replace_queue_entries` performs every insertion as a spanning-tree merge: existing entries overlapping or adjacent to the query range are deleted and re-inserted together with the new entries, and overlaps are resolved by a dominance rule —

- an inserted `Verify` or `Scanned` range always replaces what it overlaps;
- otherwise an existing `Scanned` range wins (unless a forced rescan is requested);
- otherwise the higher priority wins, and equal priorities coalesce into one range.

Gaps between disjoint ranges are filled at `Historic` priority, so the queue always tiles a contiguous span (zcash_client_sqlite, `src/wallet/scanning.rs:145--222`; zcash_client_backend, `src/data_api/scanning/spanning_tree.rs:65--153`).

3.4 Tracking the chain tip

Function `update_chain_tip` feeds the queue whenever the wallet learns a new tip height tip. Two derived heights recur, with range upper bounds exclusive throughout:

$$\mathsf{chainEnd} := \mathsf{tip} + 1, \qquad \mathsf{stable} := \mathsf{tip} - \mathsf{PRUNING\_DEPTH},$$

where PRUNING_DEPTH = 100 blocks and heights at or below stable are treated as not subject to reorganisation (zcash_client_sqlite, `src/wallet/scanning.rs:522--523,582--584`; zcash_client_sqlite, `src/lib.rs:169`). The SQLite implementation proceeds as follows (zcash_client_sqlite, `src/wallet/scanning.rs:488--660`):

(a) it is a no-op if the tip is below Sapling activation;

(b) it is a no-op if the new tip is below the prior maximum scanned height — the caller has caught the chain mid-reorganisation, and the discontinuity error path of §3.7 will resolve it;

(c) it computes a *tip shard entry* at `ChainTip` priority covering every pool's last incomplete shard,

$$\Big[\max\big(\mathsf{birthday},\ \min_{\text{pools}}\max(\mathsf{subtreeEndHeight})\big),\ \mathsf{chainEnd}\Big),$$

taking the minimum over the Sapling, Orchard, and Ironwood shards tables and emitting the entry only if that minimum lies below chainEnd;

(d) it computes a *tip entry*: with no scanned blocks, a `Historic` range from the wallet birthday to chainEnd (or `Ignored` from Sapling activation when no accounts exist); with no shard metadata (linear scanning), `Historic` from maxScanned + 1; if maxScanned > stable (steady state), `ChainTip` from maxScanned + 1 to chainEnd; otherwise a `Verify` range

$$\big[\,\mathsf{minUnscanned},\ \min(\mathsf{stable}+1,\ \mathsf{minUnscanned}+\mathsf{VERIFY\_LOOKAHEAD})\,\big),$$

where minUnscanned := maxScanned + 1.

The `Verify` discipline is documented at the same site: because `Verify` outranks everything, the connectivity check — and any rewind it forces — runs before any `ChainTip` range from this or an earlier tip update is scanned. Constant VERIFY_LOOKAHEAD = 10 blocks is chosen as roughly 12.5 minutes at 75 s per block, a window within which a user may plausibly have received a transaction without leaving the wallet open; and the `Verify` range is clamped to the stable region so that it cannot itself be reorganised away and interfere with later `Verify` ranges (zcash_client_sqlite, `src/wallet/scanning.rs:583--634`; zcash_client_sqlite, `src/lib.rs:172`).

Remark 3.7 (Two deliberate asymmetries)**.** First, when maxScanned = stable the `Verify` range above is empty: after a long offline period no `Verify` entry need exist, and connectivity is then checked only implicitly, by the previous-hash comparison of §3.7 when the adjacent range is scanned (zcash_client_sqlite, `src/wallet/scanning.rs:625--634`). Second, case (b) above means `update_chain_tip` deliberately does *nothing* when the tip moves backwards; reorganisation detection lives in the scanner, not in tip updates, and recovery in that case waits for a fetch failure or continuity error (zcash_client_sqlite, `src/wallet/scanning.rs:488--660`).

3.5 Scanning a range

Function `scan_cached_blocks` scans one contiguous range; it takes the network parameters, the block source, the wallet database, a starting height `from_height` with its prior chain state `from_state`, and an optional block limit. It panics unless fromHeight = fromState.height + 1 — the caller must supply the chain state immediately prior to the range (zcash_client_backend, `src/data_api/chain.rs:598--699`).

Keys. The scanner loads all tracked unified full viewing keys and builds a `ScanningKeys` set (`ScanningKeys::from_account_ufvks`). Per account it registers: the Sapling diversifiable full viewing key's (External, Internal) pairs of incoming viewing key and nullifier-deriving key; the Orchard full viewing key's (External, Internal) incoming viewing keys; and the *same* Orchard keys a second time under the Ironwood note-encryption domain — Ironwood notes are Orchard-shaped and decrypt under Orchard viewing keys, but carry version-3 note plaintexts, so `IronwoodDomain` is distinct from `OrchardDomain`, which accepts only version-2 plaintexts. Each incoming viewing key is tagged `(AccountId, zip32::Scope)` (zcash_client_backend, `src/scanning.rs:213--256,352--434`; the Ironwood pool is ZIP 2005, "Ironwood Quantum Recoverability", Status: Proposed, NU6.3). A scanning key derives a note's nullifier only when it carries nullifier-deriving capability: trait method `ScanningKeyOps::nf(note, position)` returns `None` for bare incoming viewing keys, so notes detected with an IVK alone never enter the tracked-nullifier set and their spends cannot be detected (zcash_client_backend, `src/scanning.rs:34--66,168--186`).

Batched trial decryption. The scanner makes two passes over the downloaded blocks. The first pass feeds every compact output into a set of `BatchRunners` — one per note-encryption domain: Sapling, Orchard, Ironwood — and flushes. A runner accumulates (domain, output) pairs across transactions and blocks into a `Batch` shared by all prepared incoming viewing keys of its domain; when the accumulated output count reaches the batch-size threshold (100 outputs in `scan_cached_blocks`) the batch is spawned on the global rayon thread pool (`rayon::spawn_fifo`) and a fresh batch begins. Results are delivered per (block hash, txid) over flume channels, and `collect_results` blocks until the relevant batch has run (zcash_client_backend, `src/scan.rs:194--220,461--626`; `src/scanning/compact.rs:101--238`; `src/data_api/chain.rs:642`). Within a batch, `zcash_note_encryption::batch::try_compact_note_decryption` batches the cryptography: `D::batch_epk` parses and prepares all ephemeral keys at once, shared secrets are computed for every (ivk, epk) pair — the scalar multiplications themselves cannot be batched — and `D::batch_kdf` derives all symmetric keys in one pass before per-output trial decryption of the 52-byte compact ciphertext (zcash_note_encryption, `src/batch.rs:42--85`); the underlying primitives are again the *Ironwood Guide*'s (§"Transmission and detection of a note: encryption, trial decryption, and synchronisation").

The second pass. The scanner then reads the prior block metadata and the wallet's unspent-nullifier set (`Nullifiers::unspent`, per pool), and calls `scan_block_with_runners` per block, accumulating summary counts, updating the in-memory nullifier set after each block (`Nullifiers::update_with`: nullifiers spent in the block are removed, nullifiers of newly received full-viewing-key-decrypted notes are added), and chaining each block's metadata into the next — so a spend of a note received earlier in the same batch is detected before anything is persisted (zcash_client_backend, `src/data_api/chain.rs:598--699`; `src/scanning.rs:436--600`).

Within a block, spend detection (`find_spent`) compares each revealed nullifier against the tracked unspent set in constant time — a subtle `CtOption` fold over the whole set; an in-code TODO notes the resulting $O(|\text{nullifiers}| \times |\text{spends}|)$ cost and questions whether constant-time operation is warranted. Nullifiers matching no tracked note are recorded as *unlinked* in a per-transaction nullifier map, in case they match a note found later when scanning an earlier range (zcash_client_backend, `src/scanning.rs:826--870`). Note detection (`find_received`) assigns each decrypted note its absolute tree position, $\text{position}(o) = \mathsf{treeSize}_{\text{tx start}} + \text{idx}(o)$, and computes a shardtree retention for every commitment in the block:

$$\text{retention}(c) = \begin{cases} \texttt{Checkpoint\{Marked\}} & \text{last in block, wallet's} \\ \texttt{Checkpoint\{None\}} & \text{last in block only} \\ \texttt{Marked} & \text{wallet's only} \\ \texttt{Ephemeral} & \text{otherwise,} \end{cases}$$

and a note is flagged `is_change` when the receiving account also spent notes in the same transaction (zcash_client_backend, `src/scanning.rs:804--824,872--994`).

Position tracking. Positions require tree sizes, which the compact stream must supply. Per pool, the tracker (`PositionTracker::for_compact_block`) reconstructs the block's starting tree size: from the prior block's metadata when available; else from the block's `ChainMetadata` final size minus the block's output count (error `TreeSizeInvalid` on underflow — the proto-default zero is caught here); else 0 below the pool's activation height (Sapling, NU5, and NU6.3 for Ironwood respectively); else `TreeSizeUnknown`. The tracker advances by each transaction's per-pool output count, and at end of block its computed size is checked against `ChainMetadata` — a mismatch is the continuity error `TreeSizeMismatch` (zcash_client_backend, `src/scanning/compact.rs:577--791`). Malformed input from the untrusted server is likewise caught up front: wrong-length nullifier bytes and malformed compact outputs or actions

yield `ScanError::EncodingInvalid` — naming pool, txid, and index — rather than a panic (zcash_client_backend, `src/scanning/compact.rs:174--234,312--390`).

Output. Each block yields a `ScannedBlock`: height, hash, block time, the wallet-relevant `WalletTx` list — a transaction is retained only if it has at least one wallet spend or output in any pool — and per-pool `ScannedBundles` holding the final tree size, the (commitment, retention) list, and the nullifier map. Transparent data in a `CompactTx` is not yet scanned (librustzcash issue #2187) (zcash_client_backend, `src/scanning/compact.rs:240--575`; `src/data_api.rs:2520--2692`). The per-call `ScanSummary` reports the scanned range and counts of spent and received Sapling and Orchard notes; received counts may include notes already spent in later blocks (a scanning-order effect), and spent counts cover only previously detected notes (zcash_client_backend, `src/data_api/chain.rs:437--502,677--685`).

Remark 3.8 (A gap in `ScanSummary`)**.** No Ironwood counters exist in `ScanSummary`, although `ScannedBlock` and `WalletTx` carry Ironwood spends and outputs; Ironwood activity is silently excluded from the summary. We read this as an oversight in the deployed code rather than a design decision (zcash_client_backend, `src/data_api/chain.rs:437--502,677--685`; `research/upstream-defects.md`, item 2: "ScanSummary lacks spent/received counters for the Ironwood pool", observed at librustzcash `e30517e433`).

The Ironwood pool otherwise traverses the pipeline as a full peer of Sapling and Orchard: its actions are trial-decrypted under `IronwoodDomain` with the account's Orchard viewing keys, its nullifiers form a separate set — method `get_ironwood_nullifiers` is a required trait method, deliberately without a panicking default because it sits on the scan path — and its commitment tree and scan-queue shard extensions are its own (zcash_client_backend, `src/scanning.rs:213--256`; `src/data_api.rs:2066--2078`).

3.6 Persistence

The scanned range is committed by a single call, `WalletWrite::put_blocks(from_state, blocks)`, which requires sequential blocks in increasing height order with `from_state` the chain state prior to the first block. The backend implementation validates, for the first block b_0 and every pool,

$$\mathsf{fromState.height}+1 = \mathsf{height}(b_0) \quad \text{and} \quad \mathsf{fromState.treeSize}+|\mathsf{commitments}(b_0)| = \mathsf{finalTreeSize}(b_0),$$

each subsequent height being the predecessor plus one; violation is `NonSequentialBlocks`. In zcash_client_sqlite the whole call runs in one database transaction (zcash_client_backend, `src/data_api.rs:3478--3490`; `src/data_api/ll/wallet.rs:291--348`; zcash_client_sqlite, `src/lib.rs:1425--1431`).

Stage 1: rows. Per block, `put_blocks` persists block metadata (heights, hash, time, per-pool final tree sizes and commitment counts); per transaction it persists transaction metadata, queues the txid for full-transaction retrieval (*enhancement*), marks spent notes, and persists received notes — checking each received note's nullifier against the *persisted* nullifier map (`detect_*_spend`) to catch notes spent in a later block range that was scanned earlier, the out-of-order counterpart of the in-memory check of §3.5. It then persists each block's unlinked-nullifier map and prunes the map beyond PRUNING_DEPTH (zcash_client_backend, `src/data_api/ll/wallet.rs:291--579,55`).

Stage 2: trees. Note commitments are assembled into shard subtrees in parallel — chunks of 1024 commitments, built at the pool's shard height, which is 16 for all three pools (subtrees of 2^{16} commitments; the Ironwood tree is Orchard-shaped) — and each shard tree is updated. The

same checkpoint set is enforced across all three trees: each tree gains a checkpoint at every height checkpointed in any other. Checkpoints at or above the NU6.3 activation height that fall on the ANCHOR_RETENTION_INTERVAL = 288-block grid are retained as durable anchors exempt from checkpoint pruning (zcash_client_backend, `src/data_api/ll/wallet.rs:597--720,1537`; `src/data_api.rs:159,166,173`). This is the boundary of the present volume: shard-tree structure, witness derivation, and anchor selection are the *Ironwood Guide*'s material (§"Proof of ownership of *some* valid note: the commitment tree").

Completing the found notes' shards. After the tree update, `scan_complete` marks the scanned range `Scanned` in the queue and, if wallet notes were found, extends the queue with `FoundNote`-priority ranges covering the blocks needed to complete each found note's subtree — the blocks that make the note witnessable. Per pool, the shard of a note at position p is

$$\mathsf{subtreeIndex}(p) = \lfloor p/2^{16} \rfloor \qquad (\texttt{Address::above\_position(SHARD\_HEIGHT, } p\texttt{)}),$$

and the extension spans from the previous shard's end height — bounded below by the wallet birthday and the pool's activation height — to the containing shard's end height plus one. The same pass marks notes `witness_stabilized` once their shard's block extent is fully `Scanned` and the shard's end height is at most the pruning floor,

$$\mathsf{floor} := \mathsf{maxScanned} - (\mathsf{PRUNING\_DEPTH} - 1)$$

(zcash_client_sqlite, `src/wallet/scanning.rs:224--473`).

Two notions of scanned height. Out-of-order scanning leaves gaps, so the backend distinguishes `block_max_scanned` — simply max(`blocks.height`) — from `block_fully_scanned`, the height to which this and all preceding blocks above the wallet birthday have been trial-decrypted. The SQLite implementation computes the latter from the queue: it is $\mathsf{end} - 1$ of the *first* `Scanned` range (by ascending start), defined only when that range starts at or below the birthday (zcash_client_backend, `src/data_api.rs:1974--1993`; zcash_client_sqlite, `src/wallet.rs:3221--3307`).

3.7 Reorganisation detection and recovery

ZIP 307 prescribes the primitive check: on each sync, re-fetch the block at the previous sync height and compare its hash with the cached copy; a mismatch means the block was orphaned (ZIP 307 §"Client-server interaction", Phases B and D). The deployed scanner generalises this into a per-block continuity check. Function `scan_block_with_runners` first runs `check_hash_continuity`, returning `ScanError::BlockHeightDiscontinuity` if $\mathsf{height}(b) \neq \mathsf{height}(\mathrm{prev}) + 1$ and `ScanError::PrevHashMismatch` if $\mathsf{prevHash}(b) \neq \mathsf{hash}(\mathrm{prev})$; for the first block of a `scan_cached_blocks` call the prior metadata is read from the wallet database (`block_metadata(from_height - 1)`), and for subsequent blocks it is the just-scanned block's own metadata (zcash_client_backend, `src/scanning/compact.rs:257--289`; `src/data_api/chain.rs:649--693`). Of ZIP 307's block-header validation list only this previous-hash check is implemented, and the ZIP's "block privacy via bucketing" enhancement — rounding the sync start down to a bucket multiple — is explicitly unimplemented (ZIP 307 §§"Block header validation", "Block privacy via bucketing").

Classification is explicit: method `is_continuity_error` on `ScanError` returns true for `PrevHashMismatch`, `BlockHeightDiscontinuity`, and `TreeSizeMismatch`; the encoding and tree-size-availability errors of §3.5 (`EncodingInvalid`, `TreeSizeUnknown`, `TreeSizeInvalid`, `TreeSizeOverflow`) are not continuity errors (zcash_client_backend, `src/scanning.rs:602--685`).

Recovery. On a continuity error the reference loop rewinds:

$$\text{rewind} := \text{errHeight} - 10 \quad (\texttt{saturating\_sub}),$$

where any height at least one block before the error is valid and the 10-block offset is a heuristic. It then calls `truncate_to_height(rewind)` on the wallet database, `truncate(rewind)` on the block cache, and returns control so the caller restarts from `update_chain_tip` and a fresh `suggest_scan_ranges` (zcash_client_backend, `src/sync.rs:423--453`). Truncation semantics — resolution to a shared shard-tree checkpoint at or below the requested height (error `RequestedRewindInvalid`, carrying a safe rewind height, otherwise), un-mining of transactions above it, and the deliberate retention of note rows — are the *Wallet Guide*'s material (§"Reorganisations: truncate and rescan"). This volume adds two consequences within its own scope: the scan queue is trimmed to the truncation height, rolling back the wallet's view of the chain tip so that the next `update_chain_tip` re-installs `Verify` and `ChainTip` ranges above the new maximum scanned height; and at the pinned commit truncation covers all three shard trees and the nullifier-map locators (zcash_client_sqlite, `src/wallet.rs:3707--3897,4252--4274`).

The depth constant. Constant PRUNING_DEPTH = 100 blocks is librustzcash's reorganisation-safety depth (zcash_client_sqlite, `src/lib.rs:169`), the nullifier-map pruning depth (zcash_client_backend, `src/data_api/ll/wallet.rs:55`), and ZIP 307's suggested witness-cache size — the ZIP asserts 100 blocks is reasonable "as no full Zcash node will roll back the chain by more than 100 blocks" (ZIP 307 §"Client operation"). The claim is grounded in zcashd: MAX_REORG_LENGTH = COINBASE_MATURITY − 1 = 99 (`src/main.h:66`), and a node asked to reorganise deeper aborts (`src/main.cpp:4332--4337`). Zebra's rollback window is MAX_BLOCK_REORG_HEIGHT = 1000, documented as local-only node policy, not consensus (`zebra-chain/src/parameters/constants.rs:20--30`), so the ZIP's "no full Zcash node" claim no longer holds for Zebra-backed deployments; PRUNING_DEPTH = 100 matches the zcashd bound, as its doc comment says (zcash_client_sqlite, `src/lib.rs:166--169`).

3.8 The reference sync loop

Module `sync` of zcash_client_backend assembles the pieces into the reference flow, whose contract the chain module documents (`src/sync.rs:58--226`, `393--467`; `src/data_api/chain.rs:1--152`):

(1–2) `update_subtree_roots`: download and store complete subtree roots for Sapling, Orchard, and Ironwood via `GetSubtreeRoots` (cf. Remark 3.2);

(3–4) `update_chain_tip` with the height from `GetLatestBlock`;

(5) `suggest_scan_ranges`;

(6) scan every `Verify` range first: download it via `GetBlockRange`, fetch the prior `ChainState` via `GetTreeState` at start − 1, scan, delete the cached blocks;

(7) scan the remaining ranges, split into chunks of the caller-supplied batch size, restarting from step (3) whenever scanning materially changed the suggestions — a new first range outranking the range just scanned, or a truncation after a continuity error (§3.7).

Blocks pass through a `BlockCache` (an extension of `BlockSource`): `get_tip_height`, `read` — short reads allowed but contiguous from the range start — `insert` — non-contiguity permitted — `delete` and `truncate`. The loop deletes each range from the cache after scanning it, the module observing that "keeping the entire chain in `CompactBlock` files on disk is horrendous for the filesystem"; deletions are spawned concurrently and awaited before the loop returns (zcash_client_backend, `src/data_api/chain.rs:237--435`; `src/sync.rs:131--226`).

The module documents its own limitations: block batches are not downloaded in parallel with scanning; detected transactions are not enhanced (the full transaction, hence the memo, is not fetched); there are no progress notifications; and there is no interruption mechanism short of process exit (zcash_client_backend, `src/sync.rs:1--10`). The chain-module documentation additionally recommends scanning `Historic` ranges in reverse height order — or from both ends inward — so that recent unspent notes are discovered sooner (zcash_client_backend, `src/data_api/chain.rs:128--134`).

4 Commitment-tree synchronisation

Scanning tells a wallet which notes it owns (§3); spending one additionally requires the note's authentication path against a recent anchor. The construction of that path — the depth-32 tree cut at half depth into height-16 shards under a height-16 cap, the wallet hashing its own shard from scanned leaves and taking every other shard as a single downloaded 32-byte root — is the *Ironwood Guide*'s material (§"Proof of ownership of *some* valid note: the commitment tree", paragraph "Derivation of the authentication path"). This section supplies what that construction presumes: the RPCs that deliver tree states and shard roots, their derivation inside the validator, and the client discipline — frontier-seeded scanning, reference-retained roots, shard-completion scan ranges — by which zcash_client_backend and zcash_client_sqlite turn a compact-block stream plus two RPCs into spendable witnesses.

4.1 The linear protocol and its limit

ZIP 307 (Status: Draft) specifies only the original, linear protocol: as compact blocks arrive in height order, the client appends every note commitment to a local depth-32 incremental Merkle tree and updates one incremental witness per unspent note; because incremental trees cannot be rewound, the ZIP advises caching roughly 100 recent tree and witness states, on the stated ground that no full node rolls back the chain by more than 100 blocks (`zips/zip-0307.rst:290--320`; the same figure resurfaces as PRUNING_DEPTH, §3.7). Under this design a wallet cannot spend a note until it has scanned every block from the note's block to the anchor block — each witness update consumes every intervening commitment — so restore-from-seed implies a complete linear scan before the first spend. Removing that spend-before-sync impossibility is the purpose of the machinery below.

Remark 4.1 (Normative status)**.** No ZIP specifies `GetTreeState`, `GetSubtreeRoots`, or the subtree-root chain index. ZIP 307's Phase A leaves tree-state acquisition an explicit unresolved TODO — "Or, it has to set X to the block height at which Sapling activated, so as to be sent the entire commitment tree. [TODO: Decide which to specify.]" (`zips/zip-0307.rst:433--439`) — and the repository holding the canonical protobufs disclaims normativity: "This repository IS not a specification of the Zcash Light Client protocol" (`lightwallet-protocol/README.md`, v0.4.1). As in Remark 3.1, we therefore cite the proto files and the deployed implementations as the specification of record, and classify the subtree-root protocol *deployed but unspecified*.

4.2 The tree-state service surface

Definition 4.2 (Tree-state and subtree-root RPCs)**.** The deployed light-wallet service defines three tree RPCs (`lightwallet-protocol/walletrpc/service.proto:307--316`):

- `GetTreeState(BlockID) returns (TreeState)` — the tree state at a block specified by height or by hash;
- `GetLatestTreeState(Empty) returns (TreeState)` — the tree state at the server's chain tip;

- `GetSubtreeRoots(GetSubtreeRootsArg) returns (stream SubtreeRoot)` — a stream of information about roots of subtrees of the note commitment tree of the requested shielded protocol — Sapling or Orchard in the deployed protocol.

[2]

Definition 4.3 (`TreeState` message)**.** Message `TreeState`, documented in the proto as "derived from the Zcash `z_gettreestate` rpc", carries

$$\begin{aligned}&\mathsf{network} : \text{string} = 1\ (\texttt{"main"} \mid \texttt{"test"}); \quad \mathsf{height} : \text{uint64} = 2;\\ &\mathsf{hash} : \text{string} = 3\ (\text{big-endian display-order hex}); \quad \mathsf{time} : \text{uint32} = 4;\\ &\mathsf{saplingTree} : \text{string} = 5; \quad \mathsf{orchardTree} : \text{string} = 6,\end{aligned}$$

where the two tree fields are protobuf *strings* — hex encodings of the legacy-serialised commitment trees of Definition 4.4 — not `bytes` (`lightwallet-protocol/walletrpc/service.proto:176--184`). At librustzcash HEAD the vendored proto adds ironwoodTree : string = 7, absent from the deployed v0.4.1 surface (Remark 3.2).

Definition 4.4 (Legacy commitment-tree encoding)**.** The tree state of one pool at one block is serialised in the legacy `CommitmentTree` wire format,

$$\mathrm{ser}(T) \;=\; \mathrm{Opt}(\mathsf{left}) \,\|\, \mathrm{Opt}(\mathsf{right}) \,\|\, \mathrm{Vec}\big(\mathrm{Opt}(\mathsf{parents}_1), \mathrm{Opt}(\mathsf{parents}_2), \ldots\big),$$

where Opt(·) writes `0x00` for an absent node and `0x01` followed by the 32-byte node otherwise, Vec(·) is CompactSize-prefixed, and each node is the 32-byte encoding of a base-field element — Jubjub base for Sapling, Pallas base for Orchard — with non-canonical encodings rejected at parse time. The parser `read_commitment_tree` bounds the parents vector at DEPTH − 1 entries, and the parsed tree converts losslessly to a frontier via `to_frontier()` (zcash_primitives, `src/merkle_tree.rs:26--61,183--209`). The producer is zcashd: the Sapling final state streams the legacy `SaplingMerkleTree` directly and the Orchard final state streams through the `OrchardMerkleFrontierLegacySer` wrapper, so both pools use the same wire format, hex-encoded (zcashd, `src/rpc/blockchain.cpp:1402--1406,1432--1436`).

Definition 4.5 (Subtree-root messages)**.** Message `GetSubtreeRootsArg` carries startIndex : uint32 = 1, the index of the first subtree to return; shieldedProtocol = 2 with `enum ShieldedProtocol { sapling = 0; orchard = 1 }` as deployed; and maxEntries : uint32 = 3, where 0 means "return all entries". Results stream in index — equivalently height — order. Message `SubtreeRoot` carries rootHash : bytes = 2, the 32-byte Merkle root of the completed subtree; completingBlockHash : bytes = 3; and completingBlockHeight : uint64 = 4. Field number 1 is absent, and no `reserved` statement documents the gap (`lightwallet-protocol/walletrpc/service.proto:186--200`).

Remark 4.6 (Byte-order warts)**.** Two byte-order pitfalls attach to these messages. First, `SubtreeRoot.completingBlockHash` is sent in big-endian display order — lightwalletd sets it to `hash32.ReverseSlice(block.Hash)` — which is the *opposite* convention from `CompactBlock.hash`, parsed into little-endian wire order; the proto documents neither convention (lightwalletd, `frontend/service.go:896--900`; `parser/block.go:59--61,115`). The reference client sidesteps the trap by ignoring `completingBlockHash` entirely, reading only `rootHash` and `completingBlockHeight` (zcash_client_backend, `src/sync.rs:247--253`). Zaino constructs the field as the block hash's `bytes_in_display_order()` (`zaino-state/src/indexer.rs:854--861`), agreeing with lightwalletd's big-endian display order; whether any

[2]The proto comment refers the reader to "section 3.7 of the Zcash protocol specification" (`service.proto:308`). The reference is stale: at the pinned spec revision, §3.4 is "Transactions and Treestates", §3.8 "Note Commitment Trees", and §3.7 "Action Transfers and their Descriptions" (`protocol/protocol.tex:4048,4248,4306`), likely a pointer into an older revision's numbering. We cite the current numbering.

deployed client consumes `completingBlockHash` remains an open question (the reference client ignores it). Second, a zcashd release note records that the serialised `finalRoot` field of `z_gettreestate` was byte-flipped relative to `getrawtransaction` output in releases up to and including v5.2.0, later rectified (zcashd, `src/rpc/blockchain.cpp:1326--1330`, help text).

4.3 Server-side derivation

Tree states. The upstream RPC is zcashd's `z_gettreestate`: it accepts a block hash or height (negative heights allowed, -1 denoting the last known valid block), requires the block to be on the main chain, and returns the block's hash, height, and time together with one object per pool, $\{\mathsf{skipHash}, \mathsf{commitments} : \{\mathsf{finalRoot}, \mathsf{finalState}\}\}$. The finalState — the serialised tree of Definition 4.4 — is present only when the block's anchor is retrievable from the coins view (`GetSaplingAnchorAt` / `GetOrchardAnchorAt`); otherwise skipHash names the most recent prior block that has one, stopping at the pool's activation height (zcashd, `src/rpc/blockchain.cpp:1303--1455`).

The proxy wraps this in a retry loop. Handler `GetTreeState` of lightwalletd rejects a request specifying neither height nor hash, marshals a height as a decimal string and a hash as big-endian hex, and calls `z_gettreestate`; if the reply's Sapling finalState is empty and its Sapling skipHash is nonempty, it re-issues the RPC at the skipHash, iterating until a final state is found or no skip hash remains, and failing with "`z_gettreestate did not return treestate`" otherwise (lightwalletd, `frontend/service.go:311--386`). The walk terminates in practice because zcashd hard-stops it at the Sapling activation height (the `saplingActive` lambda; zcashd, `src/rpc/blockchain.cpp:1408--1418`). Handler `GetLatestTreeState` calls `getblockchaininfo`, takes its `Blocks` value as the latest height, and delegates to `GetTreeState` at that height (lightwalletd, `frontend/service.go:388--401`).

Remark 4.7 (The skip walk keys on Sapling only)**.** Only the Sapling skipHash is consulted: when the walk lands on an earlier block, the returned `TreeState` carries that earlier block's height, hash, *and Orchard tree* — not the Orchard state of the requested block (lightwalletd, `frontend/service.go:311--386`).

Subtree roots. The upstream RPC is `z_getsubtreesbyindex "pool" start_index (limit)`, added in zcashd v5.6.0 and gated behind `-experimentalfeatures` + `-lightwalletd`: it returns roots of complete 2^{16}-leaf subtrees of the given pool's note commitment tree, each with end_height, the height of the block containing the note commitment that completed the subtree, reading `SubtreeData` records from the coins view (`pcoinsTip->GetSubtreeData(pool, index)`) until data is absent or the limit is reached (zcashd, `src/rpc/blockchain.cpp:1457--1539`; `doc/release-notes/release-notes-5.6.0.md:28`). The records are maintained by the validator's incremental tree: zcashd stores $\mathtt{SubtreeData}\{\mathsf{leadbyte}, \mathsf{root}, \mathsf{nHeight}\}$ and $\mathtt{LatestSubtree}\{\mathsf{leadbyte}, \mathsf{index}, \mathsf{root}, \mathsf{nHeight}\}$ records under a `uint64_t` subtree index, fixes $\mathsf{TRACKED\_SUBTREE\_HEIGHT} = 16$, and produces a completed subtree root exactly when the tree size crosses a 2^{16} boundary (zcashd, `src/zcash/IncrementalMerkleTree.hpp:20--70, 409--411`; `IncrementalMerkleTree.cpp:913--925`):

$$\mathtt{current\_subtree\_index()} = \mathsf{size}() \gg \mathsf{TRACKED\_SUBTREE\_HEIGHT}.$$

Handler `GetSubtreeRoots` of lightwalletd accepts only Sapling or Orchard and forwards to `z_getsubtreesbyindex(pool, startIndex[, maxEntries when > 0])`; for each returned subtree it fetches the block at end_height, populates the two completing-block fields from it, and streams the assembled `SubtreeRoot` (lightwalletd, `frontend/service.go:832--907`).

Example 4.8 (Mainnet Sapling subtrees 0 and 1)**.** A worked datum recorded in lightwalletd source (a quoted `z_getsubtreesbyindex` transcript, `common/common.go:178--210`): Sapling subtree 0

— outputs 0 through 65535 — has root

754bb593ea42d231a7ddf367640f09bbf59dc00f2c1d2003cc340e0c016b5b13

with end_height = 558822, and subtree 1 has root

03654c3eacbb9b93e122cf6d77b606eae29610f4f38a477985368197fd68e02d

with end_height = 670209: measured from Sapling activation at height 419,200 (zcash_protocol, src/consensus.rs:488), the first 2^{17} Sapling outputs span roughly the chain's first 250,000 post-activation blocks.

Alternative back ends. Zebra implements both z_gettreestate and z_getsubtreesbyindex in its RPC surface, so either validator can back an indexer for tree synchronisation (zebra-rpc, src/methods.rs:361,2018; src/methods/trees.rs). The successor indexer serves the same gRPC API: Zaino's get_tree_state proxies z_get_treestate to the backing validator and hex-encodes the final states (at HEAD it also populates an ironwood_tree field), its get_latest_tree_state reads the chain height and delegates, and its get_subtree_roots fetches each completing block via z_get_block(end_height) (zaino-state, src/backends/fetch.rs:1836--1896; src/indexer.rs:756--860; src/chain_index.rs:509--514, 2397--2431). One behavioural divergence: Zaino maps the proto's uint32 startIndex/maxEntries down to u16 and returns InvalidArgument when either exceeds 65535 — semantically safe, since a depth-32 tree has at most 2^{16} shards, but stricter than lightwalletd/zcashd, whose u64 subtree index yields an empty stream instead (src/chain_index.rs:509--514 versus zcashd src/zcash/IncrementalMerkleTree.hpp:20--70). Both backends delegate to the validator: NodeBackedChainIndex::get_subtree_roots calls the blockchain source (chain_index.rs:2421--2431); ValidatorConnector::State issues ReadRequest::{Sapling,Orchard,Ironwood}Subtrees to Zebra's ReadStateService, and ::Fetch proxies z_getsubtreesbyindex (chain_index/source/validator_connector.rs:545--608); only the mockchain test source self-computes (chain_index/source/mockchain_source.rs:579--665).

4.4 Client state: shards, cap, and retention

The client-side container is shardtree: "a sparse binary Merkle tree of the specified depth, represented as an ordered collection of subtrees (shards) of a given maximum height", plus a *cap* — the tree above the shard roots — and a checkpoint set enabling truncation. The root sits at address $(\text{DEPTH}, 0)$, shards at level SHARD_HEIGHT, and max_subtree_index() = $2^{\text{DEPTH}-\text{SHARD\_HEIGHT}} - 1$ (shardtree, src/lib.rs:61--129). The wallet API instantiates one such tree per pool as $\text{ShardTree}\langle \text{Store}, \text{DEPTH} = 32, \text{SHARD\_HEIGHT} = 16\rangle$ with block heights as checkpoint identifiers, via with_sapling_tree_mut / with_orchard_tree_mut (zcash_client_backend, src/data_api.rs:3743--3826). The shard height is documented as chosen to conform "to the structure of subtree data returned by lightwalletd when using the GetSubtreeRoots GRPC call": SAPLING_SHARD_HEIGHT = ORCHARD_SHARD_HEIGHT = 16, half the tree depth of each pool (src/data_api.rs:155--166). The geometry — height-16 shards of 2^{16} leaves under the 16-level cap, at most 2^{16} shards — is the *Ironwood Guide*'s construction (§"Proof of ownership of *some* valid note: the commitment tree"); the deployed constants realise it exactly (shardtree, src/lib.rs:111--129; zcash_client_sqlite, src/wallet/db.rs:1446--1447). The SQLite backend constructs each tree as $\text{ShardTree::new}(\text{store}, \text{PRUNING\_DEPTH})$ with PRUNING_DEPTH = 100: at most 100 checkpoints — one per scanned block boundary — are retained before pruning, 100 blocks being the assumed maximum reorganisation depth and ZIP 307's cache-size guidance (§4.1) (zcash_client_sqlite, src/lib.rs:169,2500,2526,2558).

What survives pruning is governed by per-node *retention* (incrementalmerkletree, src/lib.rs:76--107):

- `Ephemeral` — prunable together with its siblings;
- `Checkpoint{id, marking}` — the position is retained (its value prunable) while the checkpoint lives, decaying to `Marked`, `Reference`, or `Ephemeral` per the marking when the checkpoint is removed;
- `Marked` — retained until explicitly removed; the wallet's own note positions;
- `Reference` — retained until overwritten by a node with `Ephemeral`, `Checkpoint`, or `Marked` retention; downloaded subtree roots and seed frontiers.

4.5 Ingesting subtree roots

The reference sync driver begins *every* run by refreshing the shard roots: function `update_subtree_roots` downloads all subtree roots — the default `GetSubtreeRootsArg`, i.e. startIndex $= 0$, maxEntries $= 0$ — for Sapling and, with the `orchard` feature, Orchard, converting each streamed message by

$$\texttt{CommitmentTreeRoot::from\_parts}\big(\mathsf{completingBlockHeight},\ H\texttt{::read}(\mathsf{rootHash})\big)$$

and passing the batches to `put_sapling_subtree_roots` and `put_orchard_subtree_roots`, each with start index 0 (zcash_client_backend, `src/sync.rs:80--86,228--299`). Type `CommitmentTreeRoot` pairs the subtree end height with the root hash (`src/data_api/chain.rs:180--212`), and the trait documentation (`src/data_api.rs:3743--3826`) fixes the meaning: each value "should be the Merkle root of a subtree … containing $2^{\text{SHARD\_HEIGHT}}$ note commitments".

The SQLite backend implements both puts via one generic procedure, `put_shard_roots` parameterised by the hash H, the tree depth, and the shard height (zcash_client_sqlite, `src/wallet/commitment_tree.rs:1107--1227`):

(1) no-op on empty input;

(2) load the cap and treat it as a standalone tree of DEPTH − SHARD_HEIGHT levels whose level-0 leaves are shard roots: the `LevelShifter` wrapper offsets the hash,

$$\begin{aligned}\mathsf{emptyLeaf} &= H.\mathsf{emptyRoot}(\text{SHARD\_HEIGHT}),\\ \mathsf{combine}(\ell, a, b) &= H.\mathsf{combine}(\ell + \text{SHARD\_HEIGHT}, a, b),\end{aligned}$$

and the downloaded roots are `batch_inserted` at `Position::from`(startIndex) with `Retention::Reference` (`src/wallet/commitment_tree.rs:1122--1181`);

(3) write the cap back;

(4) check contiguity over $[\mathsf{startIndex}, \mathsf{startIndex} + n)$: `check_shard_discontinuity` rejects a call whose shard-index range is neither overlapping nor immediately adjacent to the stored $[\min(\mathsf{shardIndex}), \max(\mathsf{shardIndex}) + 1)$ range, raising `Error::SubtreeDiscontinuity` — subtree roots must arrive without gaps (`src/wallet/commitment_tree.rs:481--520`);

(5) upsert each root into `{pool}_tree_shards(shard_index, subtree_end_height, root_hash, shard_data)`; on conflict only the end height and root hash are updated, so existing scanned shard data is never clobbered, and the fresh `shard_data` written for new rows is a single root leaf with `RetentionFlags::EPHEMERAL`.

The downloaded root is cached beside any persisted subtree rather than merged eagerly — "we want to avoid deserializing the subtree just to annotate its root node, so we simply cache the downloaded root alongside of any already-persisted subtree" — and the read path resolves the pair: `get_shard` deserialises `shard_data` and applies `reannotate_root`(Some(rootHash)) (`src/wallet/commitment_tree.rs:1190--1193,412--445`).

Remark 4.9 (Idempotent full re-download)**.** The driver fixes $\mathsf{startIndex} = 0$ on every run although `put_shard_roots` supports arbitrary start indices; the upsert path makes the repetition idempotent, at the cost of re-transferring one 32-byte root (plus completing-block metadata) per completed shard per pool on every sync (zcash_client_backend, `src/sync.rs:80--86,228--299`). As of 2026-07-15 (mainnet tip 3,413,101): 1,128 complete Sapling subtrees (last completing at block 3,390,009) plus 765 Orchard (3,388,160), so each sync re-downloads 1,893 roots — 60,576 bytes of root data, 136,378 bytes of `SubtreeRoot` messages, 145,843 bytes with the five-byte gRPC frame per stream element (computed by script; counts cross-check against `ChainMetadata` at block 3,413,000: $\lfloor 73{,}932{,}454/2^{16} \rfloor = 1{,}128$, $\lfloor 50{,}191{,}264/2^{16} \rfloor = 765$). No source documents the fixed $\mathsf{startIndex} = 0$ as intentional.

4.6 Frontiers as scan seeds

A `TreeState` becomes client state through `TreeState::to_chain_state()`: it hex-decodes the block hash and reverses its bytes ("Zcashd hex strings for block hashes are byte-reversed"), and parses each pool's legacy tree (Definition 4.4) into a frontier; an *empty* tree-state string parses as the empty tree, the correct state before a pool's activation (zcash_client_backend, `src/proto.rs:367--463`). The result is

$$\texttt{ChainState}\{\,\mathsf{blockHeight};\ \mathsf{blockHash};\ \mathsf{finalSaplingTree} : \texttt{Frontier}\langle 32\rangle;\ \mathsf{finalOrchardTree} : \texttt{Frontier}\langle 32\rangle\,\},$$

"the final note commitment tree state for each shielded pool, as of a particular block height" (`src/data_api/chain.rs:504--596`; a third frontier exists at HEAD, Remark 3.2).

Every scanned batch is seeded with the chain state of the block immediately preceding it, and the seed is verified against the server's own metadata before any tree is touched:

$$\mathsf{fromHeight} = \mathsf{seed.blockHeight}+1 \quad \text{(asserted)}, \qquad \mathsf{seed.treeSize}_P + |\mathsf{cm}_P(b_0)| = \mathsf{finalTreeSize}_P(b_0)$$

per pool P, where b_0 is the batch's first block and the final tree size is read from the block's `chainMetadata`; violation yields error `NonSequentialBlocks` (zcash_client_backend, `src/data_api/chain.rs:616--635`; `src/data_api/ll/wallet.rs:291--348`). The metadata sizes are what give the scanner absolute leaf positions at all — without them it cannot even derive nullifiers (`TreeSizeUnknown`, a hard error), and a size contradicted by block contents is `TreeSizeMismatch`, a continuity error triggering the rewind of §3.7 (`lightwallet-protocol/walletrpc/compact_formats.proto:14--17,31--40`; zcash_client_backend, `src/scanning.rs:602--685`).

The batch's note commitments are then assembled into located subtrees in parallel — rayon, chunks of 1024 commitments, `LocatedTree::from_iter` at the shard level — starting at `Position::from`($\mathsf{seed.treeSize}_P$), and each pool's tree is updated by `update_tree` (zcash_client_backend, `src/data_api/ll/wallet.rs:597--771`):

(1) insert the seed frontier (`tree.insert_frontier`) with retention `Checkpoint`$\{\mathsf{id} = \mathsf{frontierHeight},\ \mathsf{marking} = \texttt{Reference}\}$ — the in-source comment: "we insert the frontier with Checkpoint retention because we need to be able to truncate the tree back to this point";

(2) `insert_tree(subtree, checkpoints)` for each built subtree;

(3) add any cross-pool checkpoints missing above the minimum retained checkpoint (`src/data_api/ll/wallet.rs:1591--1664`; the cross-pool checkpoint-unification rule itself, §3.6).

Inside shardtree, `insert_frontier` splits a non-empty frontier at the shard boundary: the leaf and the ommers below the shard level enter the shard containing the frontier's position, the

ommers at levels ≥ SHARD_HEIGHT enter the cap, checkpoint retention adds a checkpoint at the leaf position, and checkpoints beyond `max_checkpoints` are pruned. An empty frontier with checkpoint retention records a checkpoint for the empty tree (`Retention::Marked` is invalid there) (shardtree, `src/lib.rs:323--405`).

Frontier seeding is also how a wallet is born and how it rewinds. An `AccountBirthday` wraps "the chain state prior to the birthday height" plus an optional `recover_until`; `AccountBirthday::from_treestate` builds it directly from a `TreeState` for the last block *prior* to the birthday, with `height()` = priorChainState.blockHeight + 1, and account creation stores the birthday tree sizes and rewinds the wallet to the birthday chain state, so no tree information before the birthday is ever needed (zcash_client_backend, `src/data_api.rs:3010--3106`; zcash_client_sqlite, `src/wallet.rs:438--618`; §3.2). Symmetrically, `truncate_to_chain_state` inserts each pool's frontier from the target chain state with `Checkpoint{`id = targetHeight, marking = `None}`, creating the checkpoint on which the subsequent tree truncation lands (zcash_client_sqlite, `src/wallet.rs:3984--4034`; recovery flow, §3.7).

4.7 Shard completion and spendability

Two of the scan-queue priorities of §3.3 exist purely for this layer: `ChainTip` is documented as "blocks that must be scanned to complete the latest note commitment tree shard" and `FoundNote` as "blocks that must be scanned to complete note commitment tree shards adjacent to found notes" (zcash_client_backend, `src/data_api/scanning.rs:13--29`).

The tip shard. On each tip update the backend computes per pool the maximum `subtree_end_height` over the pool's shards table — the end of the last complete shard known from `GetSubtreeRoots` — takes the minimum across pools, and enqueues

$$\big[\max(\mathsf{walletBirthday},\ \mathsf{minShardTip}),\ \mathsf{tip}+1\big)$$

at `ChainTip` priority, so the incomplete tip shard of every pool is scanned even before historic ranges (zcash_client_sqlite, `src/wallet/scanning.rs:475--657`; the full tip-update case analysis, §3.4).

Found notes. After a range is scanned, `scan_complete` maps each detected note position to its shard, $s = \lfloor p/2^{16} \rfloor$ (`Address::above_position(SHARD_HEIGHT, position)`), and extends the scanned range to that shard's block extent — from the previous shard's `subtree_end_height` (the pool activation height for shard 0, clamped to the wallet birthday) to the containing shard's `subtree_end_height` + 1 — enqueuing the extensions beyond the just-scanned range at `FoundNote` priority. Completing the shard's block range is exactly what makes the note's witness computable (zcash_client_sqlite, `src/wallet/scanning.rs:224--398`).

Witness derivation. The query `witness_at_checkpoint_id(position, id)` requires position ≤ checkpoint.position() (else `QueryError::NotContained`) and folds, from the note's shard upward to the root, the root of each sibling address truncated at leaf count $\mathsf{position}_{\text{as of}} + 1$ — each sibling root drawn from the note's own shard (scanned leaves), another shard's cached root (`GetSubtreeRoots`, Definition 4.5), the cap, or a per-level empty-root constant beyond the tree size. A pruned node that cannot be replaced by an empty-root constant yields `QueryError::TreeIncomplete(addresses)` — the precise failure mode of an unready witness (shardtree, `src/lib.rs:1236--1362`; `src/error.rs:124--136`). The mathematics of the fold is the *Ironwood Guide*'s (§"Proof of ownership of *some* valid note: the commitment tree"); what this volume adds is the deployed readiness condition under which it cannot fail.

Spendability. That condition is enforced in SQL. Writing U_P for the rows of view `v_{pool}_shard_unscanned_ranges`, note selection excludes any note at position p for which some $(\mathsf{s}, \mathsf{e}, \mathsf{h}_0, \mathsf{h}_1) \in U_P$ satisfies

$$\mathsf{s} \le p < \mathsf{e} \ \wedge \ \mathsf{h}_0 \le \mathsf{anchorHeight} \ \wedge \ \mathsf{h}_1 > \mathsf{walletBirthday},$$

i.e. the note's shard has an unscanned block range at or below the anchor; the same view drives `unscanned_tip_exists`, which invalidates anchors whose containing shard is not fully scanned (zcash_client_sqlite, `src/wallet/common.rs:142--162,762--822`). The views compute $\mathsf{startPosition} = \mathsf{shardIndex} \ll 16$ and $\mathsf{endPositionExclusive} = (\mathsf{shardIndex} + 1) \ll 16$, set the shard's block extent to begin at the previous shard's end height (the pool activation height for the first shard), join the scan-queue rows whose block ranges overlap that extent, and filter to priority above `Scanned` (zcash_client_sqlite, `src/wallet/db.rs:1435--1494`). Unwinding the predicate recovers the informal statement: a note is spendable at a given anchor exactly when its own shard is scanned up to the anchor, every earlier shard is covered by a downloaded root, and the blocks from the last complete shard boundary to the anchor have been scanned (the `ChainTip` range above). Which anchor a transaction then selects, and at what confirmation depth, is the *Wallet Guide*'s material (§"The proposal"; §"Confirmations, trust, and spendability").

At the pinned librustzcash commit — part of the NU6.3 work of Remark 3.2, not of any release deployed with lightwalletd v0.4.1 — notes additionally gain a `witness_stabilized` flag once their shard's block extent is fully `Scanned` and the shard's end height has at least PRUNING_DEPTH confirmations ($\mathsf{floor} = \mathsf{maxScanned} - (\mathsf{PRUNING\_DEPTH} - 1)$); stabilised notes bypass the confirmations check at spend time, and the tip shard by definition can never stabilise (zcash_client_sqlite, `src/wallet/scanning.rs:416--473`; `src/wallet/common.rs:700--724`).

What this buys, and what remains. The linear protocol's impossibility is gone: a freshly restored wallet downloads one frontier (its birthday tree state) plus one 32-byte root per completed shard, and a note found during scanning becomes spendable as soon as its own shard's block extent and the tip range are scanned — not after the entire chain history. What remains is the trial decryption of every block from the birthday forward (§3) and the shard-completion scanning of this section; both are cost centres the *Tachyon Guide*'s proof-carrying wallet is designed to remove (§"What the fold removes", paragraphs "Trial decryption of the chain" and "Per-note witness maintenance").

5 What the server learns

A light client outsources block storage and filtering to a server it does not trust with its keys, then reconstructs its own state by trial decryption on the device (*Ironwood Guide*, §"Transmission and detection of a note: encryption, trial decryption, and synchronisation"). The keys never travel; the *requests* do. This section inventories what the deployed server-side observer learns from those requests: the stated threat model, the baseline that genuinely holds, the leaks in the deployed request pattern, the operator's instruments for recording them, and the mitigations that actually ship. Every claim about deployed behaviour cites the implementation at the commits pinned above; where the code diverges from ZIP text, we flag the divergence in place.

5.1 The stated security model

ZIP 307 names one primary goal for the light-client protocol.

Definition 5.1 (Payment detection privacy)**.** The serving proxy must not learn which transactions are addressed to a given light wallet. Under the additional assumption of network privacy ("via Tor or similar"), the proxy must also be unable to link different connections or queries as deriving from the same wallet (ZIP 307 §"Security Model").

The adversary of ZIP 307 is the node/proxy combination, modelled as *honest but curious*: it is trusted for chain-state correctness — the client accepts the block data it serves — but assumed to record and analyse everything it sees. The ZIP explicitly does not address spending notes privately (ZIP 307 §"Security Model"); transaction submission nonetheless flows through the same server, and we treat it in §5.5.

Remark 5.2 (The de facto wire specification)**.** ZIP 307 has Status Draft — it was never finalised — even though the protocol it sketches is the deployed synchronisation layer. The deployed wire format has moved past the sketch: the ZIP's `CompactBlock` carries `vtx` at field 3 and a `BlockID`, while the deployed message carries `vtx` at field 7 plus `protoVersion`, `prevHash`, `time`, an (always unset) `header`, and `ChainMetadata`; the deployed `CompactTx` adds `txid`, `fee`, Orchard actions, and transparent `vin`/`vout` counts (ZIP 307 §"Compact Stream Format" vs. lightwalletd, `lightwallet-protocol/walletrpc/compact_formats.proto`). The lightwallet-protocol proto repository, not the ZIP, is the de facto wire specification. The planned in-protocol successor is emptier still: ZIP 314, "Privacy upgrades to the Zcash light client protocol", has Status Reserved — a title with no content (`zips/zip-0314.rst`). The server implementation delegates likewise: the lightwalletd README directs developers to the external wallet app threat model for "important information about the security and privacy limitations of light wallets that use lightwalletd"; the repository itself contains no privacy analysis (lightwalletd, `README.md`).

5.2 What never leaves the device

The baseline holds. No RPC in the deployed service carries key material: across the full `CompactTxStreamer` surface (twenty methods), every request consists only of heights, hashes, txids, transparent addresses, raw transaction bytes, and pool or subtree selectors (lightwalletd, `lightwallet-protocol/walletrpc/service.proto`). Incoming viewing keys remain on the device: trial decryption of compact outputs is performed client-side by `scan_block` over locally cached `CompactBlocks` (zcash_client_backend, `src/scanning.rs`), by the mechanism of the *Ironwood Guide*, §"Transmission and detection of a note: encryption, trial decryption, and synchronisation"; witness maintenance over the resulting positions is that of §"Proof of ownership of *some* valid note: the commitment tree". Nullifier matching against the `CompactSpend` and `CompactOrchardAction` nullifier fields is likewise local.

The compact form is the reason a server mediates at all: the per-output and per-spend reductions, and the recomputed-commitment integrity substitute for the discarded AEAD tag, are those of §1 (Table 2; §1.4).

Remark 5.3 (The transparent boundary)**.** The baseline above is a shielded-layer property. The same service carries transparent-address RPCs whose requests contain cleartext addresses — `GetTaddressTxids` and `GetTaddressTransactions` (an address plus a block range), `GetTaddressBalance(Stream)` (an address list), `GetAddressUtxos(Stream)` (an address list plus a start height) — so for the transparent layer the server learns the wallet's addresses outright, by declaration rather than inference (lightwalletd, `walletrpc/service.proto`). We treat that request family as out of scope except where the shielded sync driver itself uses it (§5.3) and where deployed mitigations attach to it (§5.7).

5.3 The observable request schedule

What the server sees is a request schedule. ZIP 307 describes it in four phases: at first start, A = `GetLatestBlock` plus `GetBlock` at the tip; on each subsequent synchronisation, B = `GetLatestBlock` followed by `GetBlockRange`(X, Y) from the last-synced height X; C = `GetTransaction`(`txid`) for each transaction of interest after detection; and D = the next `GetLatestBlock` plus `GetBlockRange`(Y, Z) (ZIP 307 §"Client–server interaction").

The deployed driver refines this but does not change its shape. Per cycle, function `run` of zcash_client_backend (module `sync`, feature `sync`) issues: `GetSubtreeRoots` per shielded pool,

`GetLatestBlock`, `GetAddressUtxosStream` over *all* transparent receivers of every account (cleartext, Remark 5.3), `GetTreeState` for the block before each scan range, and `GetBlockRange` for each suggested scan range in `batch_size` chunks (zcash_client_backend, `src/sync.rs`). At the pinned commit the subtree-root fetch covers Sapling, Orchard, and Ironwood. Ironwood appears in no light-client ZIP — ZIP 2005 covers quantum recoverability and ZIP 258 only NU6.3 deployment — the request exists solely because the librustzcash proto fork defines `ShieldedProtocol.ironwood = 2` and the driver requests it (`src/sync.rs:289--296`; cf. Remark 3.1). The module's own documentation notes that this simple implementation does not itself enhance transactions; enhancement is part of the documented flow (the crate-root diagram's "Enhance transactions" step) and is driven by wallets via `transaction_data_requests` (zcash_client_backend, `src/lib.rs`, `src/sync.rs`).

The *order* of range requests is itself data. Scan ranges carry a priority (Table 5), and `suggest_scan_ranges` returns them in descending priority, then descending end height (§3.3), so Verify and ChainTip ranges near the tip are fetched first and FoundNote ranges pre-empt Historic backfill (zcash_client_backend, `src/data_api/scanning.rs`; zcash_client_sqlite, `src/wallet/scanning.rs`). The privacy consequence of that pre-emption is Remark 5.4.

5.4 Leaks in the deployed pattern

ZIP 307 asserts that compact-block synchronisation itself "leaks no information about which transactions the light client is interested in" because the compact stream is a broadcast medium (ZIP 307 §"Transaction privacy"). The same document contradicts the claim eight sections earlier, for the request pattern actually deployed: "The above interaction reveals to the server at the start of each synchronisation phase (B and D) the block height which the light client had previously synchronised to. This is an information leak under our security model" (ZIP 307 §"Block privacy via bucketing"). The broadcast argument holds only for a client that fetches everything from everyone; the deployed client fetches specific ranges at specific times and follows detections with specific txids. We take the leaks in turn.

The first request reveals the birthday. On account creation, the scan queue marks the range from Sapling activation up to the account birthday as Ignored — never downloaded — and queues scanning from the birthday itself (zcash_client_sqlite, `src/wallet.rs`). The lowest height the client ever requests therefore approximates the account's creation or recovery height, a lifetime-stable identifier of the wallet.

Each cycle reveals the prior sync height. This is the leak ZIP 307 concedes above. Its proposed mitigation is bucketing: for a bucket size N, phases B and D would request

$$\texttt{GetBlockRange}\big(X - (X \bmod N),\ Y\big) \quad \text{instead of} \quad \texttt{GetBlockRange}(X, Y),$$

hiding the exact previously-synced height within an N-block bucket (ZIP 307 §"Block privacy via bucketing"). The ZIP marks this "a proposed enhancement that has not been implemented", and the deployed code confirms: `download_blocks` issues `GetBlockRange` with start and end exactly equal to the suggested scan-range bounds, with no rounding (zcash_client_backend, `src/sync.rs`).

Detection is followed by a fetch. During compact-block scanning, function `put_blocks` records, per scanned block, exactly the subset of transactions relevant to this wallet, and queues a retrieval request for each; `transaction_data_requests` later emits Enhancement(`txid`) or GetStatus(`txid`), which the sync driver resolves via the `GetTransaction` RPC (zcash_client_backend, `src/data_api.rs`, `src/data_api/ll/wallet.rs`; zcash_client_sqlite, `src/wallet.rs`). The set of txids fetched in full therefore *equals* the wallet's own transaction set, and each fetch follows

immediately after the scan of the containing range. To the server this is a detection oracle: phase C traffic names the very transactions that payment detection privacy (Definition 5.1) is supposed to conceal, correlated in time with the range that contains them.

Here the code diverges from the ZIP, in the unusual direction: the ZIP is stricter than the deployment. ZIP 307's full-transaction mitigation is normative — the client "SHOULD obscure the exact transactions of interest by downloading numerous uninteresting transactions as well", "SHOULD download all transactions in any block from which a single full transaction is fetched", and "MUST convey to the user that fetching full transactions will reduce their privacy" (ZIP 307 §"Transaction privacy"). No decoy, chaff, or cover-traffic code exists anywhere in librustzcash or lightwalletd (negative grep over both repositories at the pinned commits), and the deployed RPC surface offers no bulk full-transaction fetch to implement the second SHOULD with — `GetBlock` returns compact data (lightwalletd, `lightwallet-protocol/walletrpc/service.proto`).

Enhancement fetches carry no timing decorrelation. The deployed request taxonomy is

$$\texttt{TransactionDataRequest} \in \{\ \mathsf{GetStatus}(\texttt{txid}),\ \mathsf{Enhancement}(\texttt{txid}),\ \mathsf{TransactionsInvolvingAddress}\{\ldots, \texttt{request\_at}, \ldots\},\ \mathsf{GetSpendingTx}(\cdot)\ \}$$

(zcash_client_backend, `src/data_api.rs`; the last two variants are feature-gated). Only TransactionsInvolvingAddress carries a `request_at` field, documented as the instant chosen so as to "decorrelate this request to a light wallet server from other requests made by the same caller", ignorable when the supplier of chain data is "private, trusted-for-privacy". The shielded-note variants Enhancement and GetStatus have no analogous jitter: the SQLite backend emits them with no `request_at` (zcash_client_sqlite, `src/wallet.rs`). The one deployed timing defence (§5.7) protects transparent address checks, not the shielded fetch-on-detect path.

Remark 5.4 (FoundNote clustering — an inferred leak)**.** When a note is found, `scan_complete` inserts scan ranges at priority FoundNote extending the scanned range so as to complete the note-commitment-tree shard(s) containing the found note, and `suggest_scan_ranges` orders by priority, so the server observes out-of-order, high-priority `GetBlockRange` fetches clustered around the blocks that contain the wallet's own notes (zcash_client_sqlite, `src/wallet/scanning.rs`; zcash_client_backend, `src/data_api/scanning.rs`). No source documents this as a leak — neither ZIP 307 nor the code documentation — and the statement above is this volume's own inference from the scan-queue code. It requires adversarial review — in particular, independent confirmation that FoundNote ranges surface to the server as distinct requests rather than being absorbed into contiguous suggested ranges — before it may be cited as fact.

Mempool polling fingerprints the session. Field `exclude_txid_suffixes` of the `GetMempoolTx` request carries suffixes of txids the client already holds, so a mempool-polling client additionally reveals which mempool transactions it has already seen, a session-linkable fingerprint (lightwalletd, `walletrpc/service.proto`). No source documents this as a leak; we record it for completeness.

5.5 The submission channel

Spending is out of ZIP 307's stated scope (§5.1), but the deployed submission path runs through the same server. The `SendTransaction` handler hex-encodes the received raw transaction and forwards it to the backing node's `sendrawtransaction` JSON-RPC (lightwalletd, `frontend/service.go`), so the operator holds the complete transaction *before* network broadcast and can link it to the submitting IP address, the TLS session, its timing, and — absent circuit isolation (§5.7) — to the same session's sync and enhancement traffic. The backing full node also sees the transaction and the `GetTransaction` txid pattern, but without client

IPs: lightwalletd terminates the client connections. The lifecycle the transaction then follows — store-then-broadcast, expiry, confirmation counting under the `ConfirmationsPolicy` — is that of the *Wallet Guide*, §"Broadcast, expiry, and the transaction lifecycle".

5.6 The operator's instruments

What the operator *can* record is a structural property, independent of any logging switch. The gRPC endpoint is operator-terminated TLS, so every connection exposes the client IP and its open and close times — a Prometheus gauge, `grpc_server_connections_current`, tracks the live connection count — and go-grpc-prometheus interceptors on both the unary and streaming paths export per-method request counts and latency histograms over the HTTP endpoint (default `127.0.0.1:9068`) (lightwalletd, `cmd/connstats.go`, `cmd/root.go`).

Logging narrows the distance between can and does. Every RPC handler logs its full request parameters at Debug level — `gRPC GetBlockRange(%+v)`, `gRPC GetTransaction(%+v)`, `gRPC SendTransaction(%+v)`, `gRPC GetTaddressBalance(%+v)`, `gRPC GetAddressUtxos(%+v)` — and full responses at Trace level (lightwalletd, `frontend/service.go`). The default level is Info (`cmd/root.go`), so a single flag, `--log-level 5`, records every block range, txid, raw transaction, and transparent address queried, to the default log file `./server.log`, without client consent or notice. A separate unary interceptor, enabled by `--grpc-logging-insecure` (default `false`; the flag name itself marks the hazard), logs {peer address, method, duration, error} per request, and carries the source comment "TODO: anonymize the addresses. cryptopan?" (lightwalletd, `common/logging/logging.go`, `cmd/root.go`).[3] The project's own architecture document is candid: "Logging may capture identifiable user data. It hasn't received any privacy analysis yet and makes no attempt at sanitization" (lightwalletd, `docs/architecture.md`).

Zaino, the successor indexer, changes none of this structurally: it serves the full `CompactTxStreamer` surface; every handler logs the method name at Info level — the literal message is `[TEST] received call`, apparent leftover test instrumentation in production code — and, under the `prometheus` feature, emits per-method request counts, duration histograms, and error counters. The metric names are `zaino.grpc.requests_total`, `zaino.grpc.request_duration_seconds`, and `zaino.grpc.errors_total`, labelled by method and code, with no peer labels; no peer-address logging exists in the dispatch path (zaino-serve, `src/rpc/grpc/service.rs`, `src/lib.rs`). Its README acknowledges the setting: "Due to current potential data leaks / security weaknesses … there is a need to use anonymous transport protocols (such as Nym or Tor) to obfuscate clients' identities from Zcash's indexing servers (Lightwalletd, Zcashd, Zaino)", and the serve crate's documentation states the ingestor "has been built so that other ingestors may be added that use different transport protocols (Nym, TOR)" — a capability anticipated, not implemented: no Nym, arti, or onion code exists in the workspace (zaino, `README.md`; zaino-serve, `src/lib.rs`; negative grep over `packages/`). Zaino is pre-1.0; all of the above is cited at commit `0e057e22` and may move.

5.7 Deployed mitigations

Two mitigations ship in the client stack. Neither touches the shielded fetch-on-detect leak of §5.4.

Transport: Tor. Crate zcash_client_backend ships an arti-based Tor client behind feature `tor`. Method `Client::connect_to_lightwalletd` tunnels the tonic gRPC channel through Tor (behind the additional feature `lightwalletd-tonic-tls-webpki-roots`), and `Client::isolated_client` returns a handle whose streams never share circuits with the parent, documented for "when you want separate parts of your program to each have a tor::Client

[3] The interceptor is wired on the unary path only, so streaming RPCs — including `GetBlockRange`, the bulk of sync traffic — never pass through it; the flag logs a skewed subset of traffic. Whether this is intentional is not determinable from the source.

handle, but where you don't want their activities to be linkable to one another over the Tor network" (zcash_client_backend, `src/tor.rs`, `src/tor/grpc.rs`). Circuit isolation is exactly the tool that would sever the submission channel of §5.5 from sync traffic. We classify this as *capability-in-librustzcash, deployment-status-unknown*: whether any deployed wallet routes `CompactTxStreamer` traffic through it by default — and whether `SendTransaction` uses a circuit isolated from sync — is not determinable from the local repositories.

Timing: exponential jitter, transparent addresses only. The SQLite backend schedules ephemeral-address checks by sampling the next check time from an exponential distribution,

$$t_{\text{next}} := t_{\text{from}} + \Delta, \qquad \Delta \sim \text{Exp}(\lambda), \qquad \lambda = 1/T, \qquad \mathbb{E}[\Delta] = T,$$

with the expected interval T set to $(24 \cdot 60 \cdot 60)/n$ seconds for n tracked ephemeral addresses — one expected check per address per day — over a shuffled address order, "so that we don't always check them in the same order" (zcash_client_sqlite, `src/wallet/transparent.rs`, `src/wallet/transparent/ephemeral.rs`). The intent is explicit in the API documentation: only one such query should be performed at a time, "to avoid linking multiple transparent addresses as belonging to the same wallet in the view of the light wallet server" (zcash_client_backend, `src/wallet.rs`, `TransparentAddressMetadata::next_check_time`). No analogous jitter exists for shielded-note enhancement (§5.4).

5.8 Summary, and the structural reading

Table 6 collects the observables.

Observable	Reveals	Mitigation status
Client IP, connection timing	wallet identity; activity windows	Tor capability in zcash_client_backend; wallet deployment unknown
Lowest `GetBlockRange` start	account birthday	none
Range start, each cycle	prior sync height	bucketing proposed (ZIP 307), unimplemented
Out-of-order FoundNote ranges	blocks holding the wallet's notes	none (inferred; Remark 5.4)
`GetTransaction` after scan	the wallet's own txids, timed	decoys normative (SHOULD), unimplemented
`exclude_txid_suffixes`	mempool transactions already seen	none
`SendTransaction`	full transaction, pre-broadcast, linked to origin	circuit-isolation capability, deployment unknown
Transparent-address RPCs	addresses outright	exponential jitter and shuffling (ephemeral addresses only)

Table 6: What the server learns from the deployed light-client protocol, and the status of each mitigation. Sources: §5.4 through §5.7.

The structural reading is that no row of the table is an implementation accident. Every leak above — IP and timing exposure, range start heights, FoundNote clustering, fetch-on-detect, submission linkage — is a property of server-mediated compact-block synchronisation itself: of detection by trial decryption over a server-supplied stream. A client that must ask for ranges reveals where it stopped; a client that must fetch what it detected reveals what it detected. Within the deployed protocol the upgrade path is a stub (ZIP 314, Reserved, Remark 5.2); the exit on

offer is architectural. The *Tachyon Guide* treats exactly this layer as the deployed protocol's removable cost centre: under the fold the wallet never scans, and services are asserted "fully oblivious to their clients' transaction details" (§"What the fold removes"), while the unresolved analogues of these same leaks — the sync-service trust model, and the timing-correlation linkage the designers call potentially fatal — carry forward as that volume's §"The sync-service trust model" and §"Timing-correlation linkage". The present section is the baseline against which those designs ask to be measured.